

What Training Moves and What It Doesn't: Cross-Lingual Mechanisms and Group-Relative Optimization in Large Language Models

Submitted in partial fulfilment of the requirements
of the degree of

Master of Science by Research

in

Data Science and Artificial Intelligence

by

Soumen Kumar Mondal

(Roll No. 23m2157)

Supervisor:

Prof. Preethi Jyothi

(Associate Professor, CSE, IIT Bombay)



Center for Machine Intelligence and Data Science (CMInDS)
INDIAN INSTITUTE OF TECHNOLOGY BOMBAY
Mumbai - 400076, India

June 26, 2026

Dedicated to my beloved parents.

Approval

This thesis entitled "**What Training Moves and What It Doesn't: Cross-Lingual Mechanisms and Group-Relative Optimization in Large Language Models**" by **Soumen Kumar Mondal** is approved for the degree of *Master of Science by Research in Data Science and Artificial Intelligence*.

.....
Prof. Preethi Jyothi
Associate Professor, CSE, IIT Bombay
(Supervisor)

.....
Prof. Biplab Banerjee
Associate Professor, CSRE, IIT Bombay
(Internal Examiner)

.....
Prof. Name
Designation, Dept, Institution
(External Examiner)

.....
Prof. Biplab Banerjee
Associate Professor, CSRE, IIT Bombay
(Chairman)

Date: June 26, 2026
Place: Mumbai, India

Declaration

This is to certify that the thesis entitled **What Training Moves and What It Doesn't: Cross-Lingual Mechanisms and Group-Relative Optimization in Large Language Models**, submitted by **Soumen Kumar Mondal** to the Indian Institute of Technology Bombay, for the award of the degree of *Master of Science by Research in Data Science and Artificial Intelligence*, is a record of the original, bona fide research work carried out by him under our supervision and guidance. The thesis has reached the standards fulfilling the requirements of the regulations related to the award of the degree.

The results contained in this thesis have not been submitted in part or in full to any other University or Institute for the award of any degree or diploma to the best of our knowledge.

.....

Prof. Preethi Jyothi

Associate Professor, CSE, IIT Bombay
(Supervisor)

Date: June 26, 2026

Place: Mumbai, India

Declaration

I declare that this written submission represents my ideas in my own words. Where others' ideas and words have been included, I have adequately cited and referenced the original source. I declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated, or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will cause disciplinary action by the Institute and can also evoke penal action from the source which has thus not been properly cited or from whom proper permission has not been taken when needed.

.....
Soumen Kumar Mondal
Roll No. 23m2157

Date: June 26, 2026
Place: Mumbai, India

Acknowledgements

I take this opportunity to acknowledge and express my gratitude to my thesis supervisor *Prof. Preethi Jyothi* who supported and guided me during the research work. I am deeply thankful to the Almighty for the abundant grace and blessings that have allowed me to make significant progress on this project.

Soumen Kumar Mondal

Roll No. 23m2157

Date: June 26, 2026

Place: Mumbai, India

Abstract

Large language models acquire much of their behaviour from data and training objectives whose effects are difficult to attribute to any single part of the network. This thesis examines two settings where this difficulty is central. The first concerns multilingual models and the extent to which their language abilities can be tied to identifiable components or shaped by fine-tuning. The second concerns preference-based optimization under competing objectives in multi-label classification.

The first part asks whether language-specific neurons can be used to improve cross-lingual transfer. Using activation-probability methods, we identify such neurons in Llama 3.1 and Mistral Nemo and test interventions on them, both at test time and through neuron-targeted LoRA fine-tuning, on XNLI and XQuAD. The interventions yield negligible gains for low-resource languages. We also find that setting activations to zero does not deactivate the neurons in any meaningful sense, and that the neurons are not independent of one another. These results suggest that language information is distributed across the network rather than confined to a small, separable set of units.

The second part studies cross-lingual behaviour during reinforcement learning. We fine-tune Qwen2.5-7B-Instruct on the MGSM benchmark with Group Relative Policy Optimization and evaluate at regular checkpoints using accuracy, logit-lens analysis across layers, and centred kernel alignment over residual representations. Most languages improve, but Telugu does not, and it is precisely Telugu whose internal representations change the most without any corresponding accuracy gain. We trace this asymmetry to tokeniser coverage rather than to language identity, arguing that a sparse vocabulary leaves the reward signal too coarse to produce a useful gradient.

The third part addresses optimization under competing objectives in multi-label classification. We propose FairPO, which splits labels into a privileged group, optimized with a preference-based objective over confusing alternatives, and a non-privileged group, kept near a reference model by a constrained objective. The two objectives are balanced through a Group Robust Preference Optimization formulation. On MS-COCO and NUS-WIDE, FairPO raises accuracy on rare labels while leaving frequent labels largely unaffected.

Contents

Approval

Declaration

Declaration

Acknowledgements

Abstract

Contents

List of Figures

List of Tables

1	Introduction	1
1.1	Motivation	1
1.2	Problem Statement	2
1.3	Research Questions	2
1.4	Research Details	3
1.4.1	Language-Specific Neurons and Cross-Lingual Transfer	3
1.4.2	Cross-Lingual Behaviour during Policy Optimization	3
1.4.3	Robust Preference Optimization for Multi-Label Classification	3
1.5	Thesis Organization	4
2	Related Work	5
2.1	Language-Specific Neurons and Internal Control of Multilingual Models	5
2.1.1	Identifying Language-Specific Neurons	5
2.1.2	Controlling and Editing Through Neurons	5
2.1.3	Limits of Neuron-Level Control	6
2.2	Cross-Lingual Processing in Language Models	6
2.2.1	The English-Centric View of Multilingual Processing	6
2.2.2	Tools for Studying Internal Representations	7

2.2.3	RL for Reasoning and Its Effect on Languages	7
2.3	MLC and Preference Optimization	7
2.3.1	Multi-Label Classification and Class Imbalance	7
2.3.2	Preference Optimization	8
2.3.3	Group-Robust Optimization	8
3	Language-Specific Neurons and Cross-Lingual Transfer	9
3.1	Introduction	9
3.1.1	Motivation	9
3.1.2	Problem Statement	10
3.1.3	Contributions	10
3.2	Background and Preliminaries	11
3.2.1	Definition of a Neuron	11
3.2.2	Notation and Problem Setup	12
3.3	Identifying Language-Specific Neurons	13
3.3.1	Activation Statistics from a Multilingual Corpus	13
3.3.2	Language Activation Probability Entropy (LAPE)	14
	Step 1: Activation probabilities.	14
	Step 2: Normalize across languages.	14
	Step 3: Entropy.	14
	Step 4: Threshold and selection.	14
3.3.3	Activation-Probability Thresholding (Act Prob 90p)	15
	Relevance score.	15
	Selection.	16
3.3.4	Language Sets	16
3.4	Intervening on Language-Specific Neurons	16
3.4.1	Test-Time Interventions	17
3.4.2	Perplexity Change as a Probe	17
3.4.3	Neuron-Targeted Fine-Tuning with LoRA	17
3.4.4	Fine-Tuning Configurations	19
3.5	Experimental Setup	19
3.5.1	Models	19
3.5.2	Tasks and Datasets	19
	XNLI.	19
	XQuAD.	19
3.5.3	Implementation Details	19
3.5.4	Evaluation Metrics	20
3.6	Results: Neuron Analysis	20
3.6.1	Number of Language-Specific Neurons	20
3.6.2	Overlap Between Languages	20
3.6.3	Layer-Wise Distribution	22
3.6.4	Perplexity Change Under Intervention	23
3.7	Results: Cross-Lingual Transfer	24

3.7.1	Baseline	25
3.7.2	Test-Time Interventions	25
3.7.3	Neuron Fine-Tuning	28
3.7.4	Random-Neuron Control	29
3.8	Discussion	31
3.8.1	Polysemantic Neurons and Entangled Function	31
3.8.2	Zero Is Not Deactivation	31
3.8.3	Neurons Are Not Independent Across Languages	32
3.8.4	The Result Does Not Depend on How Neurons Are Selected	32
3.9	Limitations	32
4	Current Research	33
5	Group Robust Preference Optimization in the Multi-Label Setting	34
5.1	Introduction	34
5.1.1	Motivation	34
5.1.2	Problem Statement	35
5.1.3	Contributions	35
5.2	The FairPO Framework	36
5.2.1	Problem Setup	36
5.2.2	Objective for the Privileged Group	36
	Confusing counterparts.	36
	Conditional objective with a fallback.	37
	DPO variant.	38
	SimPO variant.	38
	CPO variant.	38
5.2.3	Objective for the Non-Privileged Group	39
5.2.4	Group-Robust Balancing	39
5.2.5	Optimization	39
5.3	Experimental Setup	40
5.3.1	Datasets and Label Partition	40
5.3.2	Model	40
5.3.3	Baselines and Variants	41
5.3.4	Metrics and Implementation	41
5.4	Results	41
5.5	Ablation Studies	42
5.6	Discussion and Limitations	43
6	Conclusions	44
6.1	Summary of Research Contributions and Progress	44
6.2	Overall Thesis Contribution and Future Outlook	45

Bibliography	46
---------------------	-----------

List of Figures

3.1	Architecture of the MLP block in a Llama-style model. The output of self-attention is passed through a gate projection and an up projection, combined by an elementwise product after a SiLU activation, and then passed through a down projection. A neuron is one coordinate of the intermediate activation. . . .	12
3.2	Computing LAPE for a single neuron. For each language we measure the probability that the neuron activates, normalize these probabilities across languages into a distribution, and take its entropy. Neurons with low entropy activate for only a few languages.	15
3.3	Neuron-targeted fine-tuning with masked LoRA. The low-rank update $\Delta W = A \cdot B$ is masked so that the columns of B for frozen neurons are set to zero. In the example, neurons 3 and 5 are frozen with LoRA rank 2, so the corresponding columns of ΔW are zero and only the remaining neurons are updated.	18
3.4	Number of language-specific neurons identified per language by LAPE for Llama 3.1 and Mistral Nemo on Set1 and Set6. Both models assign a small set of neurons to each language.	21
3.5	Overlap between the language-specific neuron sets under Act Prob 90p, for Llama 3.1 and Mistral Nemo. The diagonal is the size of each language’s own set; the off-diagonal entries show that many neurons are shared across languages.	22
3.6	Overlap between the language-specific neuron sets under LAPE, for Llama 3.1 and Mistral Nemo on Set1 and Set6. Related languages, such as Japanese and Chinese in Set1, share the most neurons.	23
3.7	Layer-wise distribution of language-specific neurons under LAPE, for Llama 3.1 and Mistral Nemo on Set1 and Set6. The neurons concentrate in the upper layers: the higher-middle layers for Llama 3.1 and the topmost layers for Mistral Nemo.	24
3.8	Layer-wise distribution of language-specific neurons under Act Prob 90p, for Llama 3.1 and Mistral Nemo. The neurons sit at the bottom of the stack for Llama 3.1, with a large count in the first layer for English, and in the middle of the stack for Mistral Nemo.	25
3.9	Perplexity change $PPXC(i, j)$ under neuron intervention. Row i is the language whose neurons are intervened on; column j is the language whose perplexity is measured. Lower values mean the intervention has little effect. The changes are small for most interventions and are not concentrated on the diagonal.	26

5.1	Overview of the FairPO framework. Labels are partitioned into a privileged group and a non-privileged group. A sampled label is routed to the privileged path, which uses a preference loss with a BCE fallback, or the non-privileged path, which uses a constrained BCE loss against a reference model. A Group Robust Preference Optimization step balances the two group losses.	37
-----	--	----

List of Tables

3.1	Hyperparameters for neuron-targeted fine-tuning on XNLI and XQuAD. The same settings are used for Llama 3.1 and Mistral Nemo; values differ only between the two tasks.	27
3.2	No-intervention (zero-shot transfer) results for Llama 3.1 and Mistral Nemo. XNLI evaluates on vi/hi/ur (accuracy); XQuAD on vi/hi/zh (exact match with F1 in parentheses).	27
3.3	XNLI test-time intervention accuracy. No Int is the baseline; the other columns set the language-specific neuron activations to the mean, to higher percentiles, to zero, and to lower percentiles, for LAPE and Act Prob 90p. The best value in each row is in bold. No intervention improves the three target languages as a group, and the lower-percentile and zero settings often degrade accuracy, most clearly for Mistral Nemo.	28
3.4	XQuAD test-time intervention results, reported as exact match with F1 in parentheses, for the same intervention columns as Table 3.3. The pattern matches XNLI but the swings are larger: several settings collapse the score, such as Llama 3.1 with LAPE on Chinese under the mean intervention and Mistral Nemo under the higher- and lower-percentile settings. No intervention lifts the three target languages together.	29
3.5	Neuron fine-tuning results for Llama 3.1 on XNLI, for LAPE and Act Prob 90p. FTL is the fine-tuning language (English, the target, or both); EL is the evaluation language; task training data is always English. Each fine-tuned model is also evaluated under the test-time interventions (mean, $P90$, zero, $P10$). The best value in each row is in bold. No Int is the best column in nearly every row, and the FTL setting has little effect, so fine-tuning the identified neurons does not improve cross-lingual transfer.	30
3.6	Random-neuron control. Zero-shot (No Int) XNLI accuracy after fine-tuning ten randomly selected neurons per layer, instead of the language-specific neurons, for Llama 3.1 and Mistral Nemo. The scores are close to the language-specific fine-tuning results in Table 3.5, so the identified neurons are not doing something other neurons cannot.	31
5.1	Performance comparison on MS-COCO. $\mathcal{P}$ is the privileged label set (20% least frequent), and $\bar{\mathcal{P}}$ is the non-privileged set (remaining 80%). Results are mean $\pm$ std. dev. over 3 runs. The best result for each metric is in bold . ΔmAP is computed relative to the strongest baseline for each group (Focal Loss for $\mathcal{P}$, BCE-SFT for $\bar{\mathcal{P}}$). † marks a statistically significant improvement ($p < 0.05$); ‡ marks a difference that is not significant ($p > 0.1$).	42

5.2	Performance comparison on NUS-WIDE. Notation is the same as Table 5.1. . .	42
5.3	Ablation on the core components of FairPO-CPO (MS-COCO). $\Delta\text{mAP}(\mathcal{P})$ is measured against BCE-SFT. Parentheses show the change relative to the full FairPO-CPO model.	43
5.4	Ablation on the preference formulation (FairPO-CPO, MS-COCO). $\Delta\text{mAP}(\mathcal{P})$ is measured against BCE-SFT. Parentheses show the change relative to the full FairPO-CPO model.	43

Chapter 1

Introduction

1.1 Motivation

Large language models perform well on many natural language tasks, but their performance is not the same across languages. Models trained mostly on English and a few other high-resource languages do worse on low-resource languages, even though the same parameters are used for all of them [Conneau et al., 2018; Shi et al., 2023]. A second issue is that it is hard to say which part of a model is responsible for a given behaviour, and equally hard to change that behaviour by editing a specific part. An intervention that looks targeted often has little effect on the actual task.

Two lines of research address these issues from different directions. One studies the model from the inside and asks where a capability is stored. Work on language-specific neurons follows this direction, suggesting that a small set of neurons accounts for much of how a model handles a particular language, and that these neurons can be found from activation statistics [Tang et al., 2024; Kojima et al., 2024]. The other line works through the training objective, using preference-based and reinforcement-learning methods to push model outputs towards a desired signal [Rafailov et al., 2023; Shao et al., 2024]. This thesis looks at both directions together. The common observation across the studies reported here is that editing a localized part of a model is often less useful than changing the objective the model is trained on.

The problem matters for two reasons. In practice, low-resource languages and imbalanced label sets are settings where data is limited, so a method that actually moves a model’s behaviour, rather than only appearing to, is worth having. From a research point of view, the cases where an intervention fails are informative: a neuron edit that changes nothing, or a training signal that moves a representation without improving the task, tells us something about how these models are organized.

1.2 Problem Statement

This thesis studies the gap between what a model can do and what its training makes it do, in two settings: cross-lingual learning in multilingual language models, and optimization with competing objectives in multi-label classification. We break this into three sub-problems.

1. **Can localized internal components be used to control cross-lingual behaviour?** If language-specific neurons encode a language in a separable way [Tang et al., 2024; Kojima et al., 2024], then editing them should improve performance on that language. The problem is to test this on downstream tasks, not only on language of generation, and to see whether the interventions give real gains.
2. **How does reinforcement-learning fine-tuning change cross-lingual behaviour, and why does it help some languages but not others?** Training a multilingual model with a group-relative reinforcement objective [Shao et al., 2024] improves some languages and not others. The problem is to describe this difference and find the factor that controls it, rather than attributing it loosely to how low-resource a language is.
3. **How can a model be improved on a chosen subset of labels without hurting the rest?** In multi-label classification, rare or important labels are not well served by a uniform loss, but improving them naively can degrade frequent labels. The problem is to design an objective that targets the chosen labels while protecting the others.

1.3 Research Questions

From these sub-problems, the thesis investigates the following questions.

- **Are language-specific neurons useful for cross-lingual transfer?** Earlier work shows these neurons can control the *language* a model generates in [Tang et al., 2024; Kojima et al., 2024]. It is not clear whether editing them improves performance on tasks such as natural language inference [Conneau et al., 2018] and question answering [Artetxe et al., 2020] for low-resource languages.
- **Does a multilingual model process its reasoning in English at middle layers, and is that what holds back low-resource languages?** One hypothesis is that multilingual transformers work in an English-leaning space at middle layers [Wendler et al., 2024]. We ask whether this holds during reinforcement-learning fine-tuning, and whether this or some other factor explains why certain languages do not improve.
- **Can preference optimization, which was developed for text generation, be used to manage competing objectives in a classification setting?** Methods such as DPO [Rafailov et al., 2023], CPO [Xu et al., 2024b], and SimPO [Meng et al., 2024a] were built for generation. We ask whether their main ideas, together with a group-robust balancing method [Ramesh et al., 2024], can give targeted and fair improvements in multi-label classification.

1.4 Research Details

The thesis contains three studies, one for each question above. We describe them briefly here and in full in the later chapters.

1.4.1 Language-Specific Neurons and Cross-Lingual Transfer

The first study, published at the Insights from Negative Results workshop at NAACL 2025 [Mondal et al., 2025b], tests whether language-specific neurons can be used to improve cross-lingual transfer. We identify these neurons in Llama 3.1 and Mistral Nemo using Language Activation Probability Entropy and activation-probability thresholding [Tang et al., 2024], and we try two kinds of intervention: editing neuron activations at test time, and fine-tuning only those neurons with a low-rank adapter [Hu et al., 2022], on XNLI [Conneau et al., 2018] and XQuAD [Artetxe et al., 2020]. The interventions give gains of less than one accuracy point on low-resource languages. We also find that setting a neuron’s activation to zero does not turn off the behaviour assigned to it, and that the neurons found for different languages are not independent of one another. These results do not support a clean, component-level account of how a model handles language, and are consistent with a distributed encoding in which single neurons carry more than one function [Elhage et al., 2022].

1.4.2 Cross-Lingual Behaviour during Policy Optimization

The second study looks at how cross-lingual behaviour changes during reinforcement-learning fine-tuning, rather than in a fixed model. We fine-tune Qwen2.5-7B-Instruct on the MGSM benchmark [Shi et al., 2023] using Group Relative Policy Optimization [Shao et al., 2024] with a low-rank adapter, and we measure the model at regular checkpoints. The measurements cover task accuracy, the language of the model’s intermediate predictions read off with the logit lens, and the geometry of the residual-stream representations compared with centred kernel alignment. Six of the seven languages improve over training. The exception is Telugu, which has the fewest dedicated tokens in the Qwen2.5 tokeniser and whose internal representations change the most while its accuracy stays flat. We do not find the middle-layer processing in English that the latent-language hypothesis predicts [Wendler et al., 2024]. Instead, the per-language outcome follows tokeniser coverage rather than language identity: when the vocabulary cannot represent a language at a fine enough level, the reward signal is too coarse to give a useful gradient.

1.4.3 Robust Preference Optimization for Multi-Label Classification

The third study, published at the OPT workshop at NeurIPS 2025 [Mondal et al., 2025a], uses preference optimization as a tool rather than as an object of study. We propose FairPO, a method for multi-label classification that splits the labels into a privileged group and a non-privileged group. The privileged group is trained with a preference-based objective, adapted

from DPO [Rafailov et al., 2023], CPO [Xu et al., 2024b], and SimPO [Meng et al., 2024a], that makes the model prefer a true label over the incorrect labels most likely to be confused with it. The non-privileged group is kept close to a reference model by a constrained objective, which prevents its performance from dropping. The two objectives pull against each other, and we balance them with a Group Robust Preference Optimization formulation [Ramesh et al., 2024] set up as a minimax problem over the groups. On MS-COCO and NUS-WIDE, FairPO improves accuracy on rare privileged labels and leaves frequent labels almost unchanged, doing better than the baselines we compare against.

We note one point about naming. Two different methods in this thesis share the abbreviation GRPO. In Section 1.4.2 and Chapter ??, GRPO means Group Relative Policy Optimization [Shao et al., 2024], a reinforcement-learning algorithm. In Section 1.4.3 and Chapter 5, the balancing method is Group Robust Preference Optimization [Ramesh et al., 2024], which comes from the robust optimization literature. We write out the full name the first time it appears in each chapter and do not use the abbreviation on its own to tell them apart.

1.5 Thesis Organization

The rest of the thesis is organized as follows. Chapter 2 reviews related work on multilingual representations, language-specific neurons, preference optimization, and robust optimization. Chapter 3 presents the study of language-specific neurons and cross-lingual transfer. Chapter ?? presents the study of cross-lingual behaviour during Group Relative Policy Optimization. Chapter 5 presents FairPO and its results on multi-label benchmarks. Chapter ?? summarizes the findings, states their limitations, and describes directions for future work.

Chapter 2

Related Work

This chapter reviews prior work for the three studies in this thesis. We group it into three sections. Section 2.1 covers language-specific neurons and how earlier work has tried to use internal components to control multilingual behaviour. Section 2.2 covers how multilingual models process language across layers, and how this has been studied during and after RL fine-tuning. Section 2.3 covers multi-label classification, preference optimization, and group-robust optimization, which are the parts that FairPO builds on.

2.1 Language-Specific Neurons and Internal Control of Multilingual Models

2.1.1 Identifying Language-Specific Neurons

A line of work argues that a small number of neurons in a multilingual model are responsible for handling a particular language. [Tang et al. \[2024\]](#) propose Language Activation Probability Entropy (LAPE), which marks a neuron as language-specific when it activates for one or a few languages and stays quiet for the rest, and they show that these neurons sit mostly in the first and last layers of the model. [Kojima et al. \[2024\]](#) study decoder-based models and find neurons that fire for a single language, with little overlap across languages. Earlier work on encoder models reached a related conclusion: [Durrani et al. \[2020\]](#) show that small sets of neurons capture specific linguistic properties, and [Bhattacharya and Bojar \[2023\]](#) study language specificity in the feed-forward layers of transformer models. These methods differ in how they score and threshold neurons, but they share the assumption that language is, at least in part, stored in identifiable units.

2.1.2 Controlling and Editing Through Neurons

Several papers go further and use these neurons to change model behaviour. [Tang et al. \[2024\]](#) and [Kojima et al. \[2024\]](#) show that turning language-specific neurons on or off can

steer the language a model generates in. [Zhao et al. \[2024b\]](#) detect language-specific neurons with a method that needs no labelled data, and report that fine-tuning a small set of them improves performance in a target language. Related ideas appear in machine translation and style transfer: [Zhu et al. \[2024\]](#) route language-aware neurons to reduce catastrophic forgetting during translation fine-tuning, [Xie et al. \[2021\]](#) allocate neurons between shared and language-specific knowledge to improve translation, and [Lai et al. \[2024\]](#) steer text style by turning off source-style neurons. Beyond language, [Huo et al. \[2024\]](#) study domain-specific neurons in multimodal models and report accuracy gains from manipulating them. [Zhang et al. \[2024\]](#) edit factual knowledge across languages using language-agnostic factual neurons.

2.1.3 Limits of Neuron-Level Control

This control is not always reliable. A reason often given is that single neurons do not correspond to single functions. [Elhage et al. \[2022\]](#) describe superposition, where a neuron takes part in more than one feature, so editing it affects several behaviours at once. Work that traces facts across languages also suggests that knowledge is partly shared and partly transferred rather than stored in separate per-language units [[Zhao et al., 2024a](#)]. Most of the steering results above are reported on generation, that is, on which language the model writes in, and not on whether the model answers a downstream task correctly. The study in Chapter 3 [[Mondal et al., 2025b](#)] works in this gap. It tests whether neuron interventions help on XNLI [[Conneau et al., 2018](#)] and XQuAD [[Artetxe et al., 2020](#)] for low-resource languages, and finds that they do not, that setting an activation to zero does not turn the neuron off, and that neurons found for different languages are not independent.

2.2 Cross-Lingual Processing in Language Models

2.2.1 The English-Centric View of Multilingual Processing

A second group of papers studies how a multilingual model moves information across its layers when given a non-English input. [Wendler et al. \[2024\]](#) track the hidden states of Llama-2 on a word-translation task and describe three stages: the hidden states start far from any output token, then pass through a region where the model can already decode the answer but assigns more probability to its English form, and finally move to a region specific to the input language. They read this as evidence that the model works in a space closer to English at middle layers. [Zhao et al. \[2024b\]](#) propose a similar workflow, in which the model understands the input, works in English in the middle layers, and produces output in the target language at the end. [Lu et al. \[2025\]](#) look at factual recall and describe a pipeline in which the model uses an English-centric recall mechanism and then translates the English answer back into the target language; they point to two failure points, weak use of the English mechanism and wrong translation back to the target, and propose vector interventions that improve recall accuracy. Together these papers support the idea that English plays a special role inside the model, but they study translation and factual recall in fixed models rather than reasoning during training.

2.2.2 Tools for Studying Internal Representations

The studies above rely on a few standard tools. The logit lens [nostalgebraist, 2020] reads out the model’s prediction at each layer by applying the output head to intermediate hidden states, which gives a per-layer view of what the model would predict. To compare representations, Kornblith et al. [2019] introduce Centered Kernel Alignment (CKA), a similarity measure between two sets of representations that is invariant to orthogonal transformations and isotropic scaling. CKA has been used to compare representations across languages in multilingual models [Conneau et al., 2020]. The study in Chapter ?? uses both tools: the logit lens to check whether the model predicts in English at middle layers, and CKA both across languages and across training checkpoints to measure how much the representations move during fine-tuning.

2.2.3 RL for Reasoning and Its Effect on Languages

Reinforcement learning is now a common way to improve reasoning in language models. Group Relative Policy Optimization (GRPO) [Shao et al., 2024] removes the separate value model used in PPO and instead estimates the baseline from a group of sampled outputs for the same prompt. Guo et al. [2025] use GRPO to train a reasoning model with rewards based only on the correctness of the final answer. Most of this work is evaluated on English or on a mixed set of languages without separating the effect per language. Multilingual reasoning itself is measured on benchmarks such as MGSM [Shi et al., 2023], which translates grade-school math problems into several languages. A separate line of work shows that the cause of cross-lingual gaps can lie in the tokeniser: Petrov et al. [2023] show that the same text can be split into very different numbers of tokens across languages, which puts low-resource languages at a disadvantage before the model runs. Chapter ?? connects these threads. It fine-tunes a multilingual model on MGSM with GRPO, checks the English-centric processing claim during training, and finds that whether GRPO can teach a language tracks tokeniser coverage rather than language identity.

2.3 MLC and Preference Optimization

2.3.1 Multi-Label Classification and Class Imbalance

Multi-label classification usually trains a model by minimizing an aggregate loss, most often binary cross-entropy, over all labels at once. This treats every label the same, which is a problem when some labels are rare or more important than others. Charte et al. [2015a] study class imbalance in multi-label data and the resampling methods used to address it. Lin et al. [2018a] propose Focal Loss, which lowers the weight of easy examples so that training focuses on harder ones, but it still scores each label on its own and does not compare a true label against a specific competing one. Work on long-tailed multi-label recognition [Guo and Wang, 2021] and on fairness in multi-label settings [Liu et al., 2023] addresses related problems, but does not target a chosen subset of labels while protecting the rest. FairPO [Mondal et al., 2025a], the

study in Chapter 5, fills this gap by splitting the labels into a privileged group to improve and a non-privileged group to maintain. The standard benchmarks here are MS-COCO [Lin et al., 2015a] and NUS-WIDE [Chua et al., 2009a], with a Vision Transformer backbone [Dosovitskiy et al., 2021a], which FairPO also uses.

2.3.2 Preference Optimization

Preference optimization was developed to align language models from preference pairs. Direct Preference Optimization (DPO) [Rafailov et al., 2023] turns the reinforcement-learning-from-human-feedback objective into a simple classification loss over preferred and dispreferred outputs, without a separate reward model. Several reference-free variants followed. Contrastive Preference Optimization (CPO) [Xu et al., 2024b] removes the reference model and adds a negative-log-likelihood term to keep absolute quality. Simple Preference Optimization (SimPO) [Meng et al., 2024a] uses the length-normalized average log-probability as the implicit reward and adds a target margin. These methods were built for text generation, where a preference is between two full sequences. FairPO adapts their core idea, preferring a correct option over a confusing one, to per-label scores in classification, so that the model is trained to score a true label above the incorrect label most likely to be confused with it.

2.3.3 Group-Robust Optimization

The last component is the method used to balance the two groups. Distributionally Robust Optimization, and in particular Group DRO [Sagawa et al., 2020a], minimizes the worst-case loss over a set of predefined groups instead of the average loss, which improves performance on the weakest group. Group Robust Preference Optimization [Ramesh et al., 2024] brings this idea to preference learning, balancing performance across preference groups with adaptive weights in a minimax setup. FairPO uses this formulation, but defines the groups by the label partition rather than by preference source, and balances the preference-based loss for the privileged group against the constrained loss for the non-privileged group. We note again that this Group Robust Preference Optimization is a different method from the Group Relative Policy Optimization of Section 2.2.3, despite the shared abbreviation.

Chapter 3

Language-Specific Neurons and Cross-Lingual Transfer

3.1 Introduction

3.1.1 Motivation

Multilingual models handle many languages with a single set of parameters, and one open question is whether the information a model uses for a given language sits in an identifiable part of the network. Work on language-specific neurons proposes that it does. The claim is that a small number of neurons in the feed-forward layers activate mainly for one language and stay quiet for others, and that these neurons can be located from activation statistics [Tang et al., 2024; Kojima et al., 2024].

If this holds in a strong form, it points to a direct way to help low-resource languages, where models tend to do worse than on high-resource ones. The idea would be to find the neurons for a target language and change them, either at test time or through fine-tuning, so that the model handles that language better. This is the possibility we examine in this chapter.

Earlier work has shown that these neurons can control which language a model generates text in. Turning them on or off pushes the model to produce output in a given language or moves it away from that language [Tang et al., 2024; Kojima et al., 2024]. This is a result about the *form* of the output, that is, the language the model writes in. It does not say whether the neurons matter for the *content* of a task, such as deciding whether one sentence entails another or finding the answer to a question. Those tasks need the model to use what it knows, not only to write in a particular language. Whether language-specific neurons help on such tasks for low-resource languages is the question we study here.

3.1.2 Problem Statement

We study whether interventions on language-specific neurons improve cross-lingual performance on downstream tasks. By cross-lingual transfer we mean the following setting: the model is adapted using a source language, here English, for which labelled data is available, and is then evaluated on a target language for which we do not use task training data. If language-specific neurons carry the information that lets the model handle a target language, then acting on those neurons should raise target-language performance.

We make this concrete in two ways. First, we change the activations of the identified neurons at test time, replacing them with statistical values computed from a multilingual corpus, and measure the effect on the task. Second, we fine-tune only the identified neurons with a low-rank adapter, so that the update is restricted to those neurons, and again measure the effect. We run both kinds of intervention on two tasks, natural language inference with XNLI [Conneau et al., 2018] and extractive question answering with XQuAD [Artetxe et al., 2020], using two models, Llama 3.1 [Grattafiori, 2024] and Mistral Nemo [AI and NVIDIA, 2024]. We also compare two ways of identifying the neurons, so that the conclusion does not depend on a single method.

3.1.3 Contributions

This chapter, based on our published work [Mondal et al., 2025b], reports the following.

- We test two neuron identification methods, Language Activation Probability Entropy and activation-probability thresholding, on Llama 3.1 and Mistral Nemo, and two intervention types, test-time activation edits and neuron-targeted low-rank fine-tuning, on XNLI and XQuAD.
- We find that these interventions do not give consistent improvements on cross-lingual downstream tasks for low-resource languages. The change in accuracy is small, usually under one point, and is not in a consistent direction.
- We report two further observations that explain the negative result. Setting a neuron’s activation to zero does not remove its effect, which means zero is not a reliable way to turn a neuron off. And the neurons identified for different languages are not independent, so editing the neurons for one language affects others. These are consistent with neurons that take part in more than one function [Elhage et al., 2022].

The negative result here is the reason the rest of the thesis moves away from editing internal components and towards methods that act on the training objective.

3.2 Background and Preliminaries

3.2.1 Definition of a Neuron

We work with neurons in the feed-forward, or MLP, block of a transformer layer. This subsection fixes what a neuron is so that the identification and intervention methods later are precise.

Consider layer i of a transformer. Let $H_{i-1} \in \mathbb{R}^{T \times d}$ be the hidden states from the previous layer, where T is the sequence length and d is the model dimension. The layer first applies multi-head self-attention. We write its output as

$$\hat{H}_i = \text{Attn}(H_{i-1}W_q^i, H_{i-1}W_k^i, H_{i-1}W_v^i)W_o^i, \quad (3.1)$$

where $W_q^i, W_k^i, W_v^i, W_o^i \in \mathbb{R}^{d \times d}$ are the projection matrices of the attention module and $\text{Attn}(\cdot)$ is scaled dot-product attention. The MLP block takes $\hat{H}_i$ as input.

In a standard transformer MLP, the block applies an up-projection, a non-linear activation, and a down-projection:

$$h_i = \text{act_fn}\left(\hat{H}_i W_1^i\right) W_2^i, \quad (3.2)$$

where $W_1^i \in \mathbb{R}^{d \times 4d}$ and $W_2^i \in \mathbb{R}^{4d \times d}$ are the up- and down-projection matrices, and $\text{act_fn}(\cdot)$ is a non-linear activation such as GELU or SiLU. The intermediate width is commonly $4d$, though the exact factor depends on the model.

We define a neuron as one coordinate of the intermediate activation. The j -th neuron of layer i is

$$h_{i,j} = \text{act_fn}\left(\left(\hat{H}_i W_1^i\right)_j\right), \quad (3.3)$$

where $\left(\hat{H}_i W_1^i\right)_j$ is the pre-activation value of that neuron. A neuron in this sense is a single hidden unit in the MLP, and its activation is a scalar for each input token.

Recent models such as Llama use a gated variant of the MLP, the Gated Linear Unit [Shazeer, 2020]. Here an extra projection W_3^i gates the activation:

$$h_i = \left[\text{act_fn}\left(\hat{H}_i W_1^i\right) \otimes \left(\hat{H}_i W_3^i\right) \right] W_2^i, \quad (3.4)$$

where $W_3^i \in \mathbb{R}^{d \times 4d}$ and $\otimes$ is the elementwise product. The gating lets the block represent richer transformations than the plain MLP. The definition of a neuron does not change: it is still one coordinate of the intermediate activation, now formed by the gated product. Figure 3.1 shows the block. Both models in this chapter, Llama 3.1 and Mistral Nemo, use this gated form, so all neuron statistics and interventions below act on the intermediate activations of Equation 3.4.

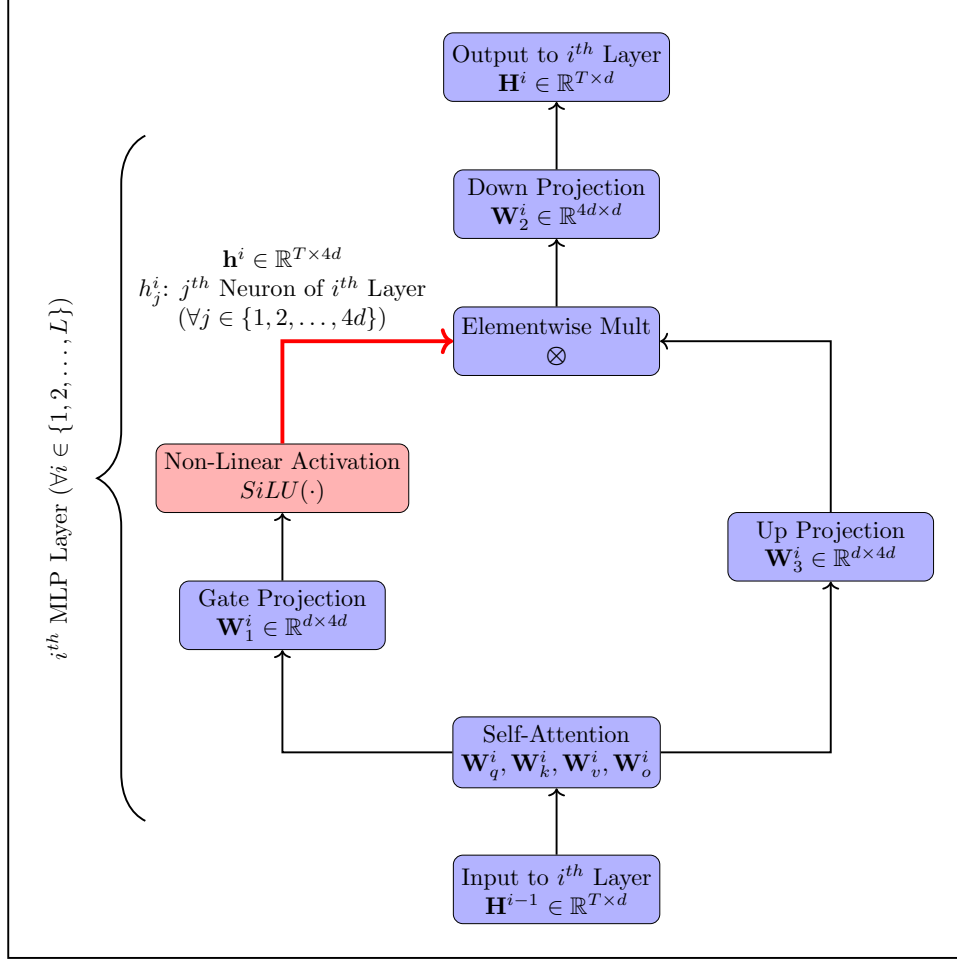


FIGURE 3.1: Architecture of the MLP block in a Llama-style model. The output of self-attention is passed through a gate projection and an up projection, combined by an elementwise product after a SiLU activation, and then passed through a down projection. A neuron is one coordinate of the intermediate activation.

3.2.2 Notation and Problem Setup

We collect the notation used in the rest of the chapter. A neuron is indexed by its layer i and position j within that layer, written (i, j) . For an input token at position t in a sequence s , the activation of neuron (i, j) is a scalar, which we write $h_{i,j}^{(s,t)}$. We compute statistics of these activations over a corpus, separately for each language, to decide which neurons are specific to a language. The details of these statistics are in Section 3.3.1.

We consider a set of languages L . For a language $l \in L$, the identification methods return a set of neurons N_l that the method marks as specific to l . An intervention is a change applied to the neurons in N_l , either replacing their activations at test time or restricting fine-tuning to them. We evaluate the effect of an intervention on a downstream task in a target language.

For the downstream tasks, we use a source language, English, for adaptation and a target language for evaluation, and we do not use task training data in the target language. This is the cross-lingual transfer setting described in Section 3.1.2. The two tasks, XNLI and XQuAD, and their evaluation metrics are defined in Section 3.5.

3.3 Identifying Language-Specific Neurons

This section describes how we find the neurons that a method marks as specific to a language. We first describe the corpus and the activation statistics we compute from it (Section 3.3.1), then the two identification methods we use, Language Activation Probability Entropy (Section 3.3.2) and activation-probability thresholding (Section 3.3.3), and finally the language sets we run them on (Section 3.3.4).

3.3.1 Activation Statistics from a Multilingual Corpus

To find language-specific neurons we first need a record of how each neuron behaves on each language. We collect this from a multilingual corpus drawn from Wikipedia [Wikimedia], using a subset of languages. Each language’s text is tokenized and passed through the model, and we record the activations from the MLP modules.

Let $h_{i,j}^t$ be the activation of neuron (i, j) at token position t in a sequence of length T . For a single sequence, the mean activation of the neuron over its tokens is

$$\bar{h}_{i,j} = \frac{1}{T} \sum_{t=1}^T h_{i,j}^t. \quad (3.5)$$

For a language l with a set of sequences $\mathcal{S}_l$, we average over all sequences to get a per-language profile of the neuron,

$$h_{i,j}^l = \frac{1}{|\mathcal{S}_l|} \sum_{s \in \mathcal{S}_l} \bar{h}_{i,j}^{(s)}. \quad (3.6)$$

Besides the mean, we compute the standard deviation and a set of quantiles of the activations for each neuron and language, since some identification methods use them. The q -quantile of a neuron’s activation is

$$P_q(h_{i,j}) = \inf \{x \in \mathbb{R} : F_{h_{i,j}}(x) \geq q\}, \quad (3.7)$$

where $F_{h_{i,j}}$ is the cumulative distribution function of the neuron’s activations. We compute the 5th, 10th, 25th, 50th, 75th, 90th, and 95th percentiles. These statistics, computed once per language, are the input to the identification methods below and also to the test-time interventions in Section 3.4.

3.3.2 Language Activation Probability Entropy (LAPE)

The first method is Language Activation Probability Entropy (LAPE) [Tang et al., 2024]. The idea is that a neuron is specific to a language if it tends to activate for that language and not for others. LAPE measures this with the entropy of a neuron’s activation probabilities across languages: a low entropy means the neuron activates for one or a few languages, while a high entropy means it activates evenly across all of them.

Step 1: Activation probabilities. For a language l , the activation probability of neuron (i, j) is the fraction of tokens for which the neuron has a positive activation,

$$p_{i,j}^l = \mathbb{E} [\mathbb{I}(h_{i,j}^l > 0)], \quad (3.8)$$

where $\mathbb{I}(\cdot)$ is the indicator function and the expectation is over all tokens in all sequences for language l .

Step 2: Normalize across languages. For a set of k languages, we normalize the probabilities so that they form a distribution over languages for each neuron,

$$\mathcal{P}_{i,j}^l = \frac{p_{i,j}^l}{\sum_{l'=1}^k p_{i,j}^{l'}}. \quad (3.9)$$

Step 3: Entropy. The LAPE value of a neuron is the entropy of this distribution,

$$\text{LAPE}(i, j) = - \sum_{l=1}^k \mathcal{P}_{i,j}^l \log \mathcal{P}_{i,j}^l. \quad (3.10)$$

A small LAPE value marks a neuron that activates for few languages. Figure 3.2 shows the steps.

Step 4: Threshold and selection. A neuron is only considered if it activates often enough for at least one language. We set a threshold at the 95th percentile of the activation probabilities over all neurons and languages,

$$\text{Threshold} = \text{Quantile}_{0.95}(p_{i,j} \mid \forall i, j, l), \quad (3.11)$$

and keep neuron (i, j) only if $p_{i,j}^l > \text{Threshold}$ for some language l . Among the neurons that pass, we select the ones with the lowest LAPE values. We take the bottom 1% of all neurons,

$$m = \frac{1}{100} \times 4d \times L, \quad (3.12)$$

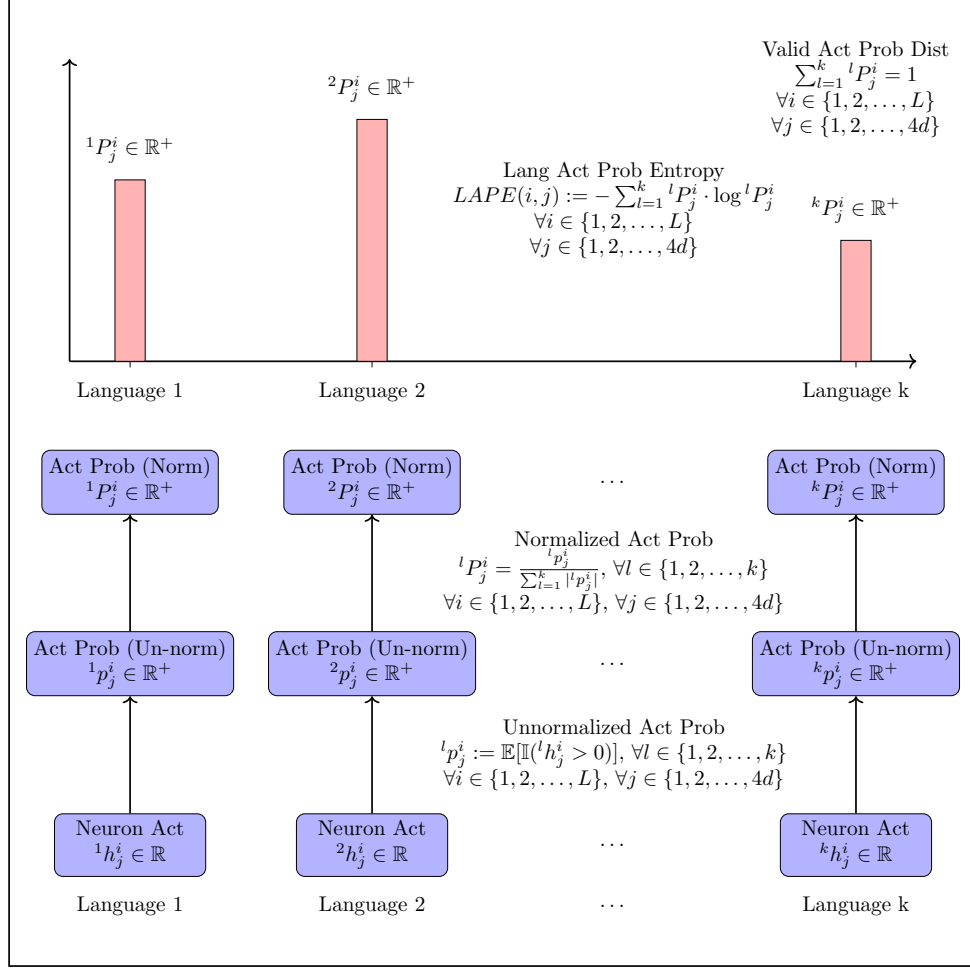


FIGURE 3.2: Computing LAPE for a single neuron. For each language we measure the probability that the neuron activates, normalize these probabilities across languages into a distribution, and take its entropy. Neurons with low entropy activate for only a few languages.

where L is the number of layers and $4d$ the number of neurons per layer. Each selected neuron is then assigned to the languages for which it activates above the threshold, which gives the per-language neuron sets N_l .

3.3.3 Activation-Probability Thresholding (Act Prob 90p)

The second method scores each neuron by how often it activates strongly for a language, and keeps the top-scoring neurons per language. We use this as a check against LAPE, so that the conclusion does not rest on one identification method. We refer to it as Act Prob 90p, after the relevance score we use.

Relevance score. For each neuron and language we compute a relevance value $r_{i,j}^l$. Several choices are possible. The simplest is the activation probability, the fraction of tokens with a

positive activation,

$$r_{i,j}^l = \mathbb{E} [\mathbb{I}(h_{i,j}^l > 0)] . \quad (3.13)$$

We can also use the mean absolute activation, the probability of exceeding the layer mean, or the probability of exceeding the 90th-percentile activation. The last of these is the score we use in the main experiments,

$$r_{i,j}^l = \mathbb{E} [\mathbb{I}(h_{i,j}^l > P90_{i,j}^l)] , \quad (3.14)$$

where $P90_{i,j}^l$ is the 90th percentile of the neuron’s activations for language l . This score picks out neurons that have rare but strong activations, which we take as a sign of a specialized response.

Selection. For each language l we collect the relevance values into a tensor $R^l = \{r_{i,j}^l\}$ and keep the top m neurons by relevance. Here we use

$$m = \frac{0.125}{100} \times L \times 4d, \quad (3.15)$$

and the per-language set is

$$N_l = \{(i, j) \mid (i, j) \in \text{Top}_m(R^l)\} . \quad (3.16)$$

LAPE and Act Prob 90p differ in how they score and select neurons, so if both lead to the same conclusion about interventions, the result is less likely to be an artifact of one method.

3.3.4 Language Sets

We run the identification on two groups of languages, which we call Set1 and Set6. Set1 is a mixed group covering several scripts and language families, used to study neurons across languages that the model handles with varying success. Set6 is a group of Indian languages, used to look more closely at the low-resource case that is the focus of this chapter. We compute the activation statistics and run both identification methods separately for each set, since the set of languages changes the normalization in LAPE (Equation 3.9) and therefore which neurons are selected. The exact languages in each set are listed in Section 3.5.

3.4 Intervening on Language-Specific Neurons

Having identified the neurons, we now change them and measure the effect. We use two kinds of intervention. The first replaces neuron activations at test time (Section 3.4.1). The second fine-tunes only the identified neurons (Section 3.4.3). We also describe a perplexity-based probe (Section 3.4.2) that we use to check whether a test-time intervention is doing anything to the model’s handling of a language.

3.4.1 Test-Time Interventions

In a test-time intervention we run the model as usual, but for the neurons in N_l we replace the activation that the model would have produced with a fixed value taken from the statistics in Section 3.3.1. The value is the same for every token, and the rest of the model runs unchanged. This lets us ask whether forcing the language-specific neurons to a particular level changes the model’s behaviour on the target language.

We try several replacement values. The mean activation μ for the language sets the neuron to its typical level. The higher percentiles, $P75$, $P90$, and $P95$, push the neuron towards strong activation. The lower percentiles, $P25$, $P10$, and $P5$, push it towards weak activation. We also try setting the activation to zero, which is often described as turning the neuron off. As we show later (Section 3.7), setting the activation to zero is not the same as removing the neuron’s effect, because zero is not a neutral value for these activations.

3.4.2 Perplexity Change as a Probe

Before measuring downstream task accuracy, it helps to check whether an intervention affects the model’s basic handling of a language at all. We use perplexity for this. If the neurons in N_l matter for language l , then intervening on them should change the model’s perplexity on text in language l .

We define the perplexity change $\text{PPXC}(i, j)$ as the difference between the perplexity on language j after intervening on the neurons identified for language i and the perplexity with no intervention,

$$\text{PPXC}(i, j) = \text{PPX}(j \mid \text{intervention on } i) - \text{PPX}(j). \quad (3.17)$$

A value near zero means the intervention barely touches the model’s handling of language j . A large value means the intervention has a strong effect. Comparing the diagonal entries ($i = j$, intervening on a language’s own neurons) with the off-diagonal entries ($i \neq j$, intervening with another language’s neurons) shows how specific the neurons are to their own language.

3.4.3 Neuron-Targeted Fine-Tuning with LoRA

The second intervention fine-tunes the identified neurons rather than fixing their activations. We use Low-Rank Adaptation (LoRA) [Hu et al., 2022], which adds a low-rank update ΔW to a weight matrix while leaving the pretrained weight W unchanged. The update is factored as

$$\Delta W = A \cdot B, \quad (3.18)$$

with $A \in \mathbb{R}^{d_{\text{in}} \times r}$ and $B \in \mathbb{R}^{r \times d_{\text{out}}}$, where the rank r is much smaller than d_{in} or d_{out} .

To restrict the update to the identified neurons, we apply a binary mask to the factors,

$$A = \text{mask}_A \cdot A_t, \quad B = \text{mask}_B \cdot B_t, \quad (3.19)$$

where A_t and B_t are the trainable matrices. We leave mask_A all ones, and in mask_B we set to zero the columns that correspond to neurons we want to freeze. For a frozen neuron index j_f ,

$$[\text{mask}_B]_{i,j} = \begin{cases} 0, & \text{if } j = j_f, j_f \in \text{Frozen Neurons}, \\ 1, & \text{otherwise.} \end{cases} \quad (3.20)$$

This makes $\Delta W_{i,j} = 0$ for every frozen neuron, so no update reaches it. The fine-tuned MLP output is then

$$y = x \cdot W + x \cdot \Delta W, \quad (3.21)$$

where only the selected neurons receive an update through ΔW . Figure 3.3 illustrates the masking for a small example.

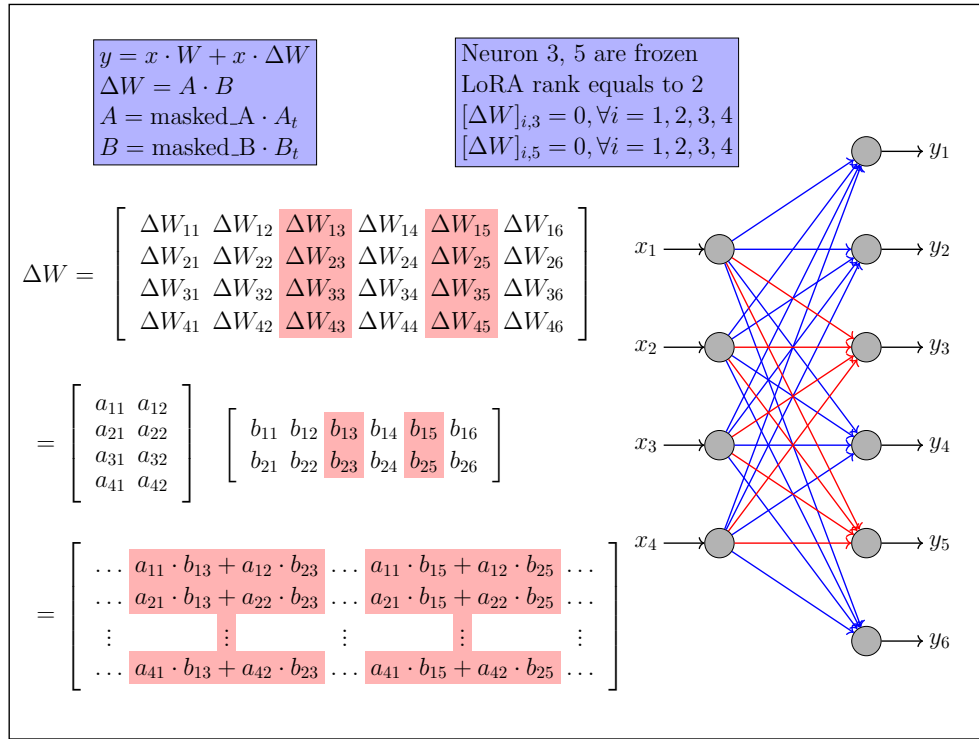


FIGURE 3.3: Neuron-targeted fine-tuning with masked LoRA. The low-rank update $\Delta W = A \cdot B$ is masked so that the columns of B for frozen neurons are set to zero. In the example, neurons 3 and 5 are frozen with LoRA rank 2, so the corresponding columns of ΔW are zero and only the remaining neurons are updated.

Alongside the masked MLP update, we always fine-tune the attention projections (q, k, v, o) and the task-specific head with ordinary LoRA. This keeps the task performance reasonable, so that the comparison is between updating the identified neurons and not updating them, rather than between training and not training at all.

3.4.4 Fine-Tuning Configurations

We vary which language’s neurons are fine-tuned. We refer to this as the fine-tuning language (FTL). In one setting we fine-tune the neurons identified for the source language, English. In another we fine-tune the neurons for the target language. In a third we fine-tune the union of both. In all cases the task training data is English, since we are testing cross-lingual transfer and do not use target-language task data. Comparing these settings shows whether it matters which language’s neurons are made trainable, which is a direct test of whether the language-specific sets carry transferable information.

3.5 Experimental Setup

3.5.1 Models

We use two pretrained multilingual models. Llama 3.1 [Grattafiori, 2024] is an 8-billion-parameter model from Meta. Mistral Nemo [AI and NVIDIA, 2024] is a 12-billion-parameter model. Both use the gated MLP of Equation 3.4, so the neuron definition and the identification methods apply to both without change. We load both models in 4-bit precision to reduce memory use.

3.5.2 Tasks and Datasets

XNLI. The Cross-lingual Natural Language Inference (XNLI) benchmark [Conneau et al., 2018] tests whether a model can decide the relationship between two sentences across languages. Given a premise and a hypothesis, the model labels the pair as entailment, neutral, or contradiction. We use English as the source language and evaluate on lower-resource target languages from our language sets. We form the input by joining the premise and hypothesis with a separator token and tokenize to a maximum length of 512 tokens.

XQuAD. The Cross-lingual Question Answering Dataset (XQuAD) [Artetxe et al., 2020] tests extractive question answering across languages. Given a passage and a question, the model must select the span of the passage that answers the question. As with XNLI, we adapt using English and evaluate on the target languages, without using target-language task data.

3.5.3 Implementation Details

For neuron identification we collect activations on the Wikipedia corpus [Wikimedia] with a maximum sequence length of 512 tokens. For fine-tuning we use LoRA on the MLP neurons as described in Section 3.4.3, together with LoRA on the attention projections and the task head. We train with the AdamW optimizer and mixed-precision computation. The full list of

hyperparameters, including LoRA rank, scaling factor, learning rate, number of epochs, and batch size for each task, is given in Appendix ??.

3.5.4 Evaluation Metrics

For XNLI we report classification accuracy on the three-way label. For XQuAD we report exact match and F1 between the predicted span and the reference answer. For the neuron analysis we report the perplexity change PPXC of Equation 3.17, along with counts and overlaps of the identified neurons. The no-intervention numbers in Section 3.7 serve as the baseline against which all interventions are compared.

3.6 Results: Neuron Analysis

Before looking at task performance, we study the neurons that the two methods identify. This tells us how many neurons are marked per language, how much the sets overlap across languages, where they sit in the model, and whether intervening on them changes the model’s handling of a language at all. We run this analysis on the two language sets, Set1 and Set6, described in Section 3.3.4.

3.6.1 Number of Language-Specific Neurons

Figure 3.4 shows the number of neurons identified per language by LAPE for both models. For both models, the Act Prob 90p method identifies a fixed number of language-specific neurons regardless of the language: 573 neurons per language for the first model and 717 for the second. The counts are a small fraction of the total number of neurons, which is expected given the selection rules in Equations 3.12 and 3.15. The two methods do not pick the same number of neurons for a given language, since they score and threshold differently, but both assign a comparable, small set to each language. This sets up the rest of the analysis: the neurons are few enough that, if they carried language-specific function in a clean way, intervening on them should be both possible and visible.

3.6.2 Overlap Between Languages

If a neuron were specific to one language, it would appear in the set for that language and no other. To check this, we count how many neurons are shared between each pair of languages. Figure 3.5 shows the overlap for Act Prob 90p, and Figure 3.6 shows it for LAPE. In each heatmap the diagonal is the size of a language’s own set, and an off-diagonal entry is the number of neurons two languages have in common.

The sets are far from disjoint. Under Act Prob 90p, where each language has 573 neurons in Llama 3.1 and 717 in Mistral Nemo, the shared counts between pairs of languages often run to

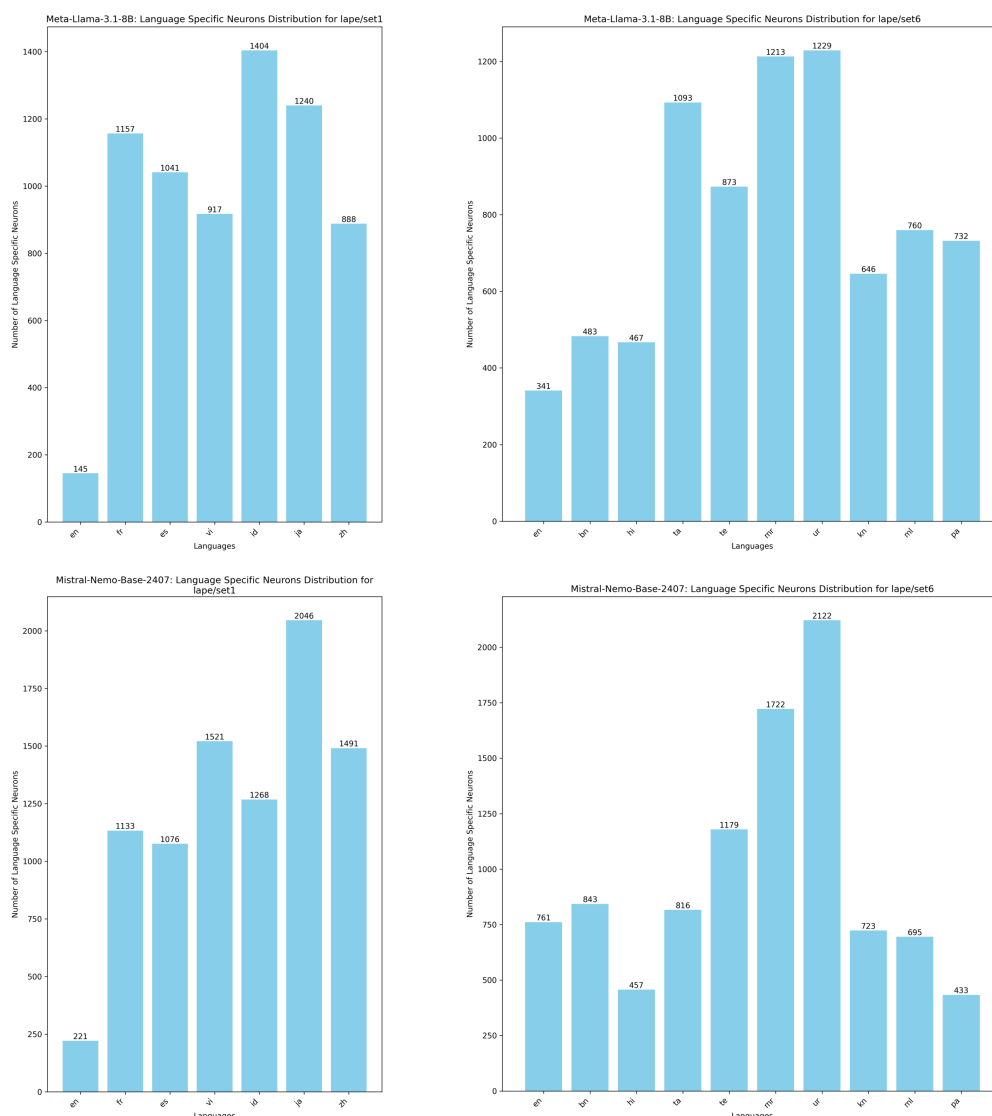


FIGURE 3.4: Number of language-specific neurons identified per language by LAPE for Llama 3.1 and Mistral Nemo on Set1 and Set6. Both models assign a small set of neurons to each language.

a few hundred, so a sizable part of any one set is not unique to that language. LAPE shows the same thing on its larger sets. The amount of sharing also tracks how close the languages are: pairs such as Japanese and Chinese in Set1 overlap much more than unrelated pairs, and the Indian languages in Set6 overlap most among themselves.

This is the first sign that the identified neurons are not cleanly tied to a single language, and it matters for the interventions that follow. If a neuron belongs to several languages at once, it cannot be changed for one of them without affecting the others, so there is no clean way to act on a single target language through these sets.

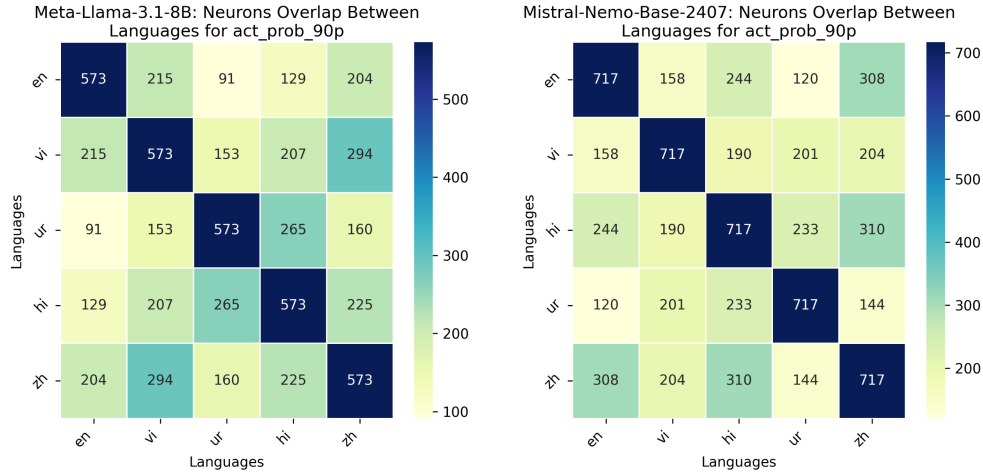


FIGURE 3.5: Overlap between the language-specific neuron sets under Act Prob 90p, for Llama 3.1 and Mistral Nemo. The diagonal is the size of each language’s own set; the off-diagonal entries show that many neurons are shared across languages.

3.6.3 Layer-Wise Distribution

We next look at where the identified neurons sit across the layers of the model. Figure 3.7 shows the distribution for LAPE, and Figure 3.8 shows it for Act Prob 90p. The location of the neurons depends on both the method and the model, and the two methods do not agree.

Under LAPE, the neurons sit mostly in the upper part of the stack. For Llama 3.1 the counts are highest in the higher-middle layers, around layers 24 to 30, for both Set1 and Set6. For Mistral Nemo the neurons concentrate in the topmost layers, around layers 37 to 39, where the counts are much larger than anywhere else. The bottom layers hold few neurons under LAPE, apart from a small number of isolated cells.

Under Act Prob 90p the picture is different, and it also differs between the two models. For Llama 3.1 the neurons are pushed towards the bottom of the stack: the first layer holds a large number of English neurons, there is a strong band around layers 3 and 4 across all languages, and the deeper layers stay sparse. For Mistral Nemo the neurons sit in the middle of the stack, roughly layers 9 to 20, rather than at either end, with the highest counts around layers 10 to 13 and layer 19.

So the two methods place language-specific neurons in different parts of the model, and the layer pattern is not the same across the two models either. We point this out because it matters for the discussion. As Section 3.7 shows, the two methods lead to the same downstream conclusion even though they select neurons in different layers, which means the result does not depend on where in the model the neurons are taken from.

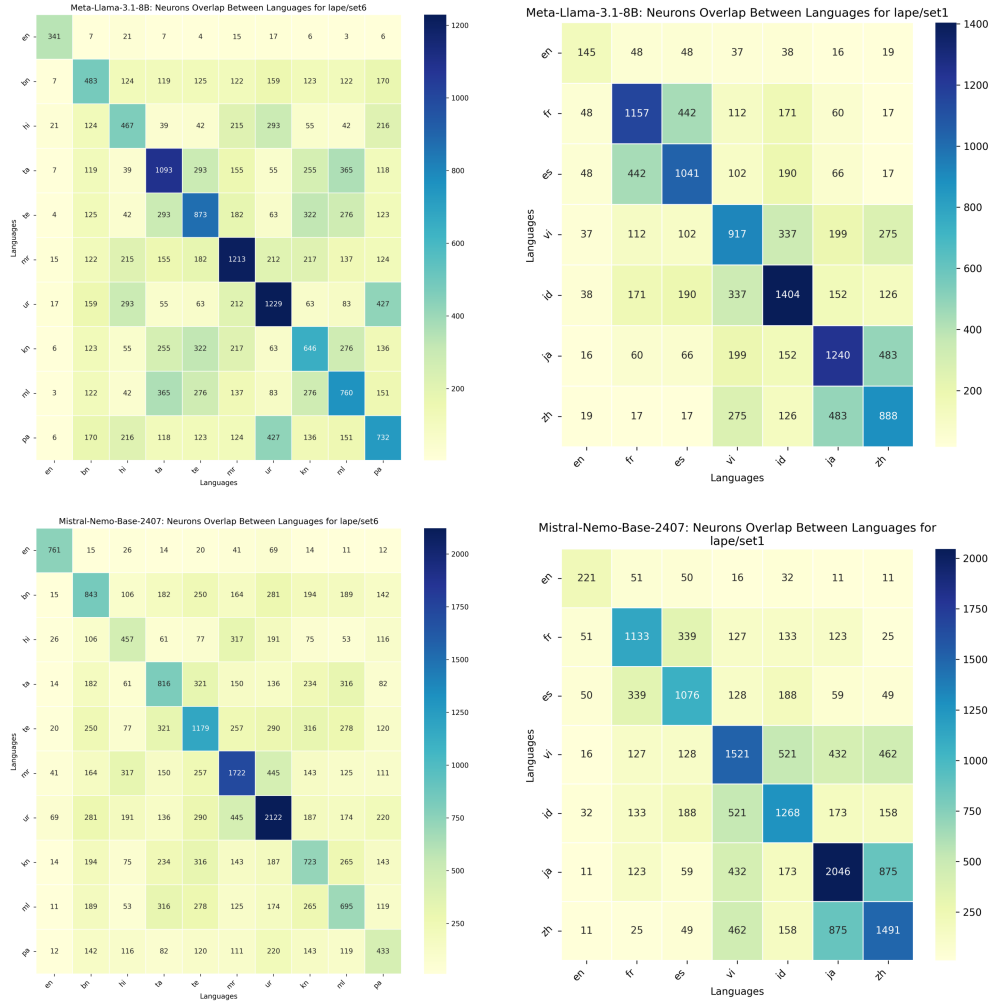


FIGURE 3.6: Overlap between the language-specific neuron sets under LAPE, for Llama 3.1 and Mistral Nemo on Set1 and Set6. Related languages, such as Japanese and Chinese in Set1, share the most neurons.

3.6.4 Perplexity Change Under Intervention

We now check whether intervening on the neurons changes the model’s handling of a language, using the perplexity change of Equation 3.17. Figure 3.9 shows $\text{PPXC}(i, j)$ as a heatmap, where row i is the language whose neurons are intervened on and column j is the language whose perplexity is measured. If the neurons were specific to their own language, the diagonal ($i = j$) would be large and the off-diagonal entries small.

The heatmaps do not show this clean pattern. For most interventions the perplexity change is small, including on the diagonal, which means that intervening on a language’s own neurons often does little to the model’s perplexity on that language. Where there are larger changes, they are not confined to the diagonal. Setting activations to zero, in particular, does not produce the large diagonal effect that would be expected if zero turned the neurons off. We take this up again in the discussion, since it bears directly on the common practice of treating zero as a way to deactivate a neuron.

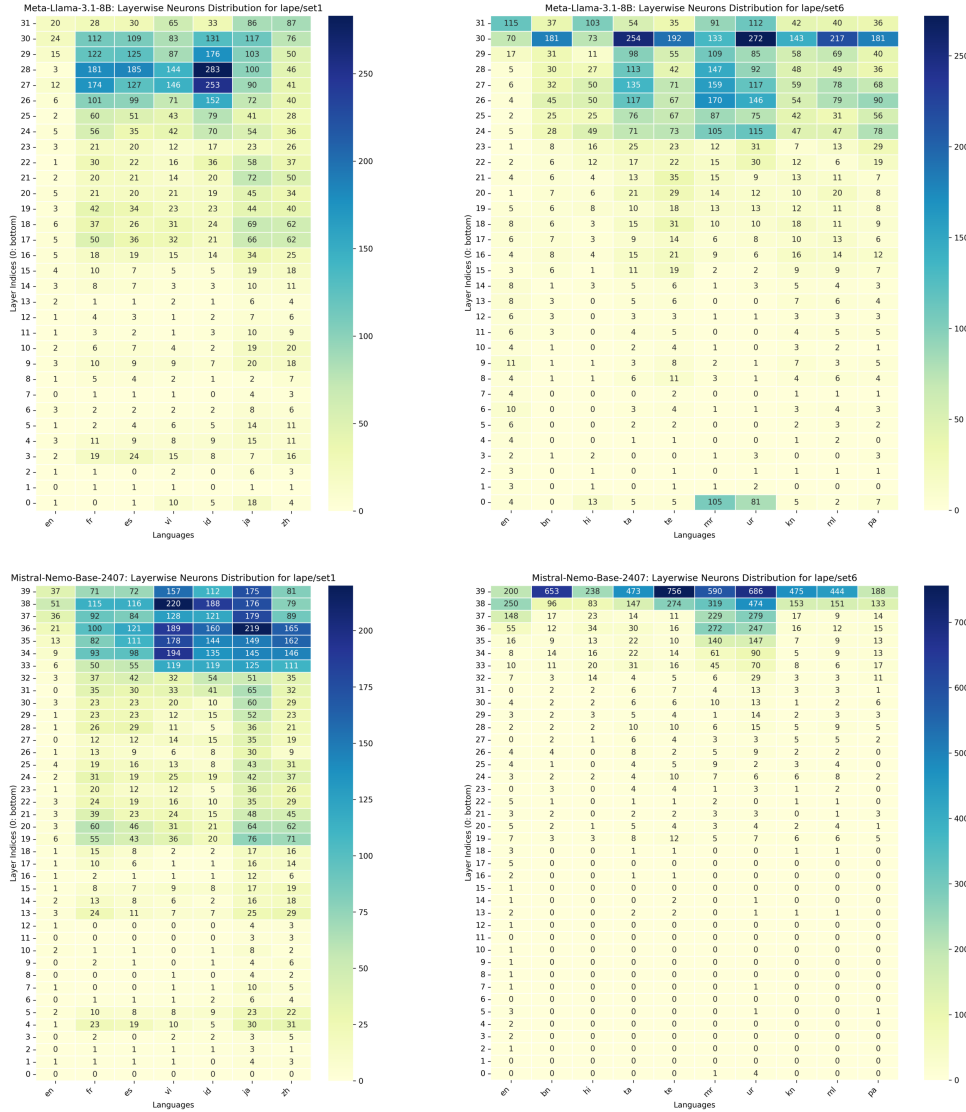


FIGURE 3.7: Layer-wise distribution of language-specific neurons under LAPE, for Llama 3.1 and Mistral Nemo on Set1 and Set6. The neurons concentrate in the upper layers: the higher-middle layers for Llama 3.1 and the topmost layers for Mistral Nemo.

3.7 Results: Cross-Lingual Transfer

We now turn to the main question, whether the interventions improve cross-lingual task performance. We report results on XNLI and XQuAD, using English as the source language and lower-resource languages as targets. For XNLI the target languages are Vietnamese (vi), Hindi (hi), and Urdu (ur). For XQuAD they are Vietnamese (vi), Hindi (hi), and Chinese (zh). In all settings the model is adapted using English task data and evaluated on the target language without target-language task data. The hyperparameters used for all fine-tuning runs are listed in Table 3.1.

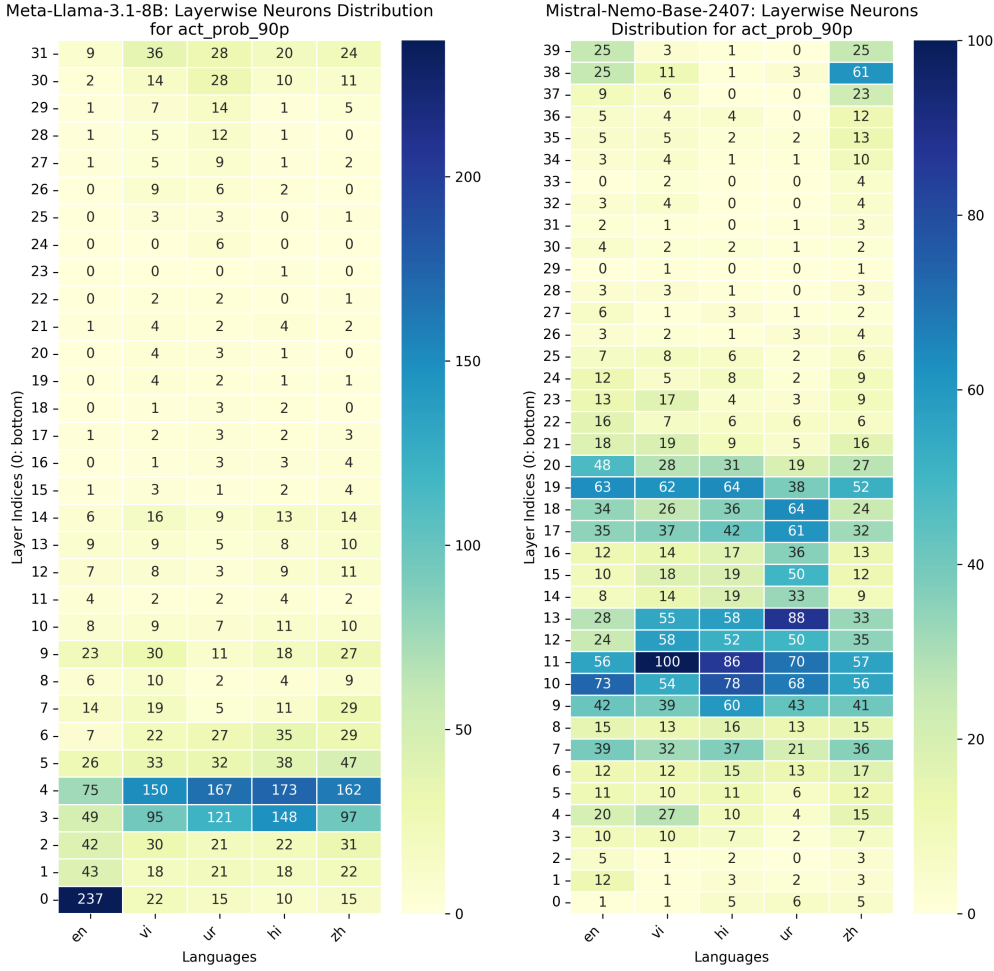


FIGURE 3.8: Layer-wise distribution of language-specific neurons under Act Prob 90p, for Llama 3.1 and Mistral Nemo. The neurons sit at the bottom of the stack for Llama 3.1, with a large count in the first layer for English, and in the middle of the stack for Mistral Nemo.

3.7.1 Baseline

Before applying any intervention, we record the no-intervention performance of each model, which is the zero-shot cross-lingual transfer after adaptation on English with no neuron edited. Table 3.2 reports these numbers. They are the reference for the rest of the section: an intervention is only useful if it improves on this baseline. For XNLI we report accuracy on Vietnamese, Hindi, and Urdu; for XQuAD we report exact match and, in parentheses, F1 on Vietnamese, Hindi, and Chinese.

3.7.2 Test-Time Interventions

Tables 3.3 and 3.4 report the test-time intervention results on XNLI and XQuAD. In each table the baseline is the No Int column, and the remaining columns replace the language-specific

es	0.05	0.29	0.31	0.56	0.36	1.34	2.96
fr	0.06	1.27	0.42	0.32	0.28	0.29	0.47
de	0.07	0.36	1.05	0.23	0.25	0.26	0.46
vi	0.07	0.22	0.19	4.74	0.72	0.44	1.20
id	0.05	0.22	0.27	0.43	2.17	0.24	0.39
zh	0.11	0.17	0.18	0.40	0.27	31.40	9.21
zh	0.09	0.12	0.20	0.41	0.28	4.90	11.52
	es	fr	de	vi	id	zh	zh

FIGURE 3.9: Perplexity change $PPXC(i, j)$ under neuron intervention. Row i is the language whose neurons are intervened on; column j is the language whose perplexity is measured. Lower values mean the intervention has little effect. The changes are small for most interventions and are not concentrated on the diagonal.

neuron activations with the mean, with higher percentiles ($P75$, $P90$, $P95$), with zero, and with lower percentiles ($P5$, $P10$, $P25$), for both identification methods.

The main point is that no intervention improves the target languages as a group. On XNLI, the best column for most rows is No Int, and where an intervention edges above the baseline the gain is small, for example Llama 3.1 with LAPE on Hindi and Urdu, where the mean intervention adds about two tenths of a point. These small gains do not carry across languages or across the two models, so they read as noise rather than a real effect.

The interventions do not only fail to help, they often hurt, and the size of the damage depends on the model, the task, and the replacement value. For Llama 3.1 the changes on XNLI stay within a few points. For Mistral Nemo they are much larger: the lower-percentile and zero settings can pull accuracy down sharply, and with Act Prob 90p the $P5$ setting on Vietnamese drops from 80.5 to the high thirties. The lower percentiles are the most destructive overall, which fits the idea that forcing these neurons to a low value disturbs computations the task depends on.

XQuAD shows the same pattern more strongly. The scores swing widely across intervention columns, and several settings collapse the model: the mean intervention takes Llama 3.1 with LAPE on Chinese from 56 exact match to 10, and for Mistral Nemo the higher-percentile and low-percentile settings drive exact match to 0 on some languages. As on XNLI, No Int is at

Setting	XNLI	XQuAD
<i>Models</i>		
Backbones	Meta-Llama-3.1-8B, Mistral-Nemo-Base-2407	
Quantization	4-bit (nf4), double quantization	
Compute dtype	bfloat16	
<i>LoRA</i>		
Rank r	64	64
Scaling factor α	128	128
<i>Optimization</i>		
Optimizer	AdamW	AdamW
β_1, β_2	0.95, 0.999	0.95, 0.999
Learning rate	1×10^{-6}	5×10^{-5}
Weight decay	0.1	0.1
Gradient clipping	10.0	10.0
Warm-up	1% of steps	1% of steps
LR schedule	linear decay	linear decay
<i>Training</i>		
Epochs	2	10
Batch size	8	4
Max context length	256	512
Train split used	25%	100%
Eval split used	100%	100%

TABLE 3.1: Hyperparameters for neuron-targeted fine-tuning on XNLI and XQuAD. The same settings are used for Llama 3.1 and Mistral Nemo; values differ only between the two tasks.

Model	Eval Lang	XNLI (Acc %)	XQuAD (EM (F1))
Llama 3.1	vi	80.5	41 (73.5)
	hi	75.0	38 (64.1)
	ur / zh	70.0	56 (77.5)
Mistral Nemo	vi	80.5	39 (74.6)
	hi	76.1	38 (66.9)
	ur / zh	66.8	47 (74.9)

TABLE 3.2: No-intervention (zero-shot transfer) results for Llama 3.1 and Mistral Nemo. XNLI evaluates on vi/hi/ur (accuracy); XQuAD on vi/hi/zh (exact match with F1 in parentheses).

or near the best column for most rows, and no intervention lifts the three target languages together.

Model & Method	Lang	No Int	Int μ	P75	P90	P95	Int 0	P5	P10
Llama 3.1 + LAPE	vi	80.5	79.5	79.1	79.0	78.9	79.8	73.6	77.7
	hi	75.0	75.2	74.9	74.9	75.0	74.4	74.8	75.1
	ur	70.0	70.4	70.2	69.3	69.0	68.5	68.6	68.7
Llama 3.1 + Act Prob 90p	vi	80.5	78.2	78.9	79.3	79.5	79.0	76.5	77.4
	hi	75.0	74.1	72.6	71.8	71.2	73.7	75.0	74.6
	ur	70.0	69.7	69.7	69.3	68.7	69.6	69.4	69.5
Mistral Nemo + LAPE	vi	80.5	80.4	80.5	80.6	80.5	79.2	80.6	80.5
	hi	76.1	69.8	67.8	66.9	66.6	74.9	73.0	72.4
	ur	66.8	66.5	67.2	67.0	66.8	66.9	65.8	65.4
Mistral Nemo + Act Prob 90p	vi	80.5	67.4	79.0	81.1	81.1	79.8	37.4	40.7
	hi	76.1	72.2	73.7	74.5	74.4	74.5	62.4	66.3
	ur	66.8	65.9	64.0	61.3	59.7	66.4	54.6	61.6

TABLE 3.3: XNLI test-time intervention accuracy. No Int is the baseline; the other columns set the language-specific neuron activations to the mean, to higher percentiles, to zero, and to lower percentiles, for LAPE and Act Prob 90p. The best value in each row is in bold. No intervention improves the three target languages as a group, and the lower-percentile and zero settings often degrade accuracy, most clearly for Mistral Nemo.

Setting the activations to zero is worth singling out, since zeroing is often used as a way to switch a neuron off. The Int 0 column does not behave like removing a needed component. On XNLI it stays close to the baseline in most rows, and on XQuAD it is sometimes among the better intervention columns rather than the worst. This matches the perplexity result in Section 3.6.4: zero is just another value these activations can take, not an off state. The two identification methods, LAPE and Act Prob 90p, lead to the same overall picture, so the outcome does not depend on how the neurons are chosen.

3.7.3 Neuron Fine-Tuning

The test-time interventions in Section 3.7.2 fixed the neuron activations to a chosen value. Here we instead fine-tune the identified neurons, so the model can adjust them through training rather than having them set by hand. Table 3.5 reports this for Llama 3.1 on XNLI, for both identification methods and under the three fine-tuning-language (FTL) settings from Section 3.4.4: fine-tuning the source-language (English) neurons, the target-language neurons, or both. The task training data is always English. For each fine-tuned model we also apply the test-time interventions, so each row shows the no-intervention score next to the mean, *P*90, zero, and *P*10 settings.

Two things stand out. First, the no-intervention column is the best column in nearly every row, for both LAPE and Act Prob 90p. Fine-tuning the neurons and then intervening on them does not beat simply fine-tuning and leaving the neurons alone. Second, the choice of which

Model & Method	Lang	No Int	Int μ	P75	P90	P95	Int 0	P5	P10
Llama 3.1 + LAPE	vi	41 (73.5)	40 (72.9)	36 (76.7)	31 (69.5)	28 (66.8)	32 (69.2)	4 (35.8)	10 (43.2)
	hi	38 (64.1)	40 (65.5)	38 (66.4)	36 (65.4)	36 (66.3)	23 (49.9)	38 (62.8)	37 (62.8)
	zh	56 (77.5)	10 (62.8)	3 (58.1)	3 (56.1)	4 (52.9)	33 (63.2)	20 (49.1)	33 (63.2)
Llama 3.1 + Act Prob 90p	vi	41 (73.6)	39 (73.0)	26 (65.8)	23 (64.9)	24 (64.8)	42 (73.8)	33 (67.8)	36 (70.3)
	hi	38 (64.1)	34 (60.7)	35 (62.3)	36 (62.8)	32 (61.9)	38 (62.9)	23 (54.9)	31 (58.6)
	zh	56 (77.5)	61 (80.7)	59 (79.5)	56 (78.8)	56 (78.4)	55 (78.5)	40 (61.7)	50 (73.6)
Mistral Nemo + LAPE	vi	39 (74.6)	42 (76.8)	40 (76.4)	40 (75.0)	39 (73.6)	13 (45.0)	10 (40.4)	11 (41.2)
	hi	38 (66.9)	35 (65.9)	38 (67.1)	37 (66.6)	34 (66.5)	22 (51.7)	36 (65.8)	36 (66.1)
	zh	47 (74.9)	24 (74.0)	4 (65.6)	0 (61.6)	0 (59.3)	14 (53.3)	1 (33.6)	24 (68.9)
Mistral Nemo + Act Prob 90p	vi	39 (74.6)	11 (43.3)	26 (62.8)	29 (63.9)	15 (48.5)	39 (74.5)	0 (3.8)	0 (6.5)
	hi	38 (66.9)	26 (54.4)	34 (65.1)	37 (68.9)	37 (68.7)	33 (63.8)	0 (6.0)	0 (11.9)
	zh	47 (74.9)	46 (77.4)	26 (69.9)	20 (59.8)	15 (54.9)	48 (76.2)	0 (8.5)	0 (17.0)

TABLE 3.4: XQuAD test-time intervention results, reported as exact match with F1 in parentheses, for the same intervention columns as Table 3.3. The pattern matches XNLI but the swings are larger: several settings collapse the score, such as Llama 3.1 with LAPE on Chinese under the mean intervention and Mistral Nemo under the higher- and lower-percentile settings. No intervention lifts the three target languages together.

language’s neurons are made trainable barely changes the result. Within a target language the three FTL settings give almost the same scores, for example Vietnamese stays near 80.1 under LAPE whether we fine-tune the English neurons, the Vietnamese neurons, or both. If the language-specific neurons carried information that transfers to the target language, we would expect fine-tuning the target-language neurons to help more than fine-tuning the English ones, and we do not see that.

The one place any column edges above the baseline is Urdu, where the mean intervention reaches about 70.4 to 70.6 against a baseline near 70.0. This is a fraction of a point, it appears only for one language, and it does not hold for Vietnamese or Hindi, so it is not evidence that the approach works. The pattern is the same under Act Prob 90p as under LAPE. Since Act Prob 90p does not depend on the choice of language set, while LAPE does, the agreement between the two also rules out the language-set normalization in LAPE as the reason for the negative result. Overall the fine-tuning experiment points the same way as the test-time one: acting on the identified neurons does not improve cross-lingual transfer.

3.7.4 Random-Neuron Control

The interventions so far act on neurons that a method marked as language-specific. A natural question is whether these neurons matter more than any other neurons. To check this, we run a control in which we fine-tune randomly chosen neurons instead of the identified ones, keeping the number of fine-tuned neurons the same (ten per layer). Everything else, including the English task training and the evaluation setup, is unchanged. Table 3.6 reports the zero-shot (No Int) accuracy after this random-neuron fine-tuning.

FTL	EL	No Int	Int μ	Int P90	Int 0	Int P10
<i>Llama 3.1 with LAPE</i>						
en	vi	80.2	79.6	78.5	79.2	78.0
vi	vi	80.1	79.5	78.6	79.2	78.0
en+vi	vi	80.1	79.4	78.5	79.1	78.0
en	hi	74.9	74.6	74.6	74.1	74.6
hi	hi	74.9	74.6	74.5	74.3	74.6
en+hi	hi	74.9	74.5	74.5	74.3	74.7
en	ur	69.8	70.4	69.6	70.2	69.0
ur	ur	69.8	70.5	69.5	70.4	69.1
en+ur	ur	70.0	70.6	69.5	70.3	68.9
<i>Llama 3.1 with Act Prob 90p</i>						
en	vi	80.3	78.0	79.1	78.5	77.5
vi	vi	80.1	78.0	79.0	78.7	77.4
en+vi	vi	80.1	78.0	78.9	78.8	77.4
en	hi	74.9	73.9	72.1	73.3	74.5
hi	hi	74.9	73.8	72.1	73.9	74.6
en+hi	hi	74.9	73.7	72.1	73.8	74.6
en	ur	69.9	70.1	69.4	70.0	69.5
ur	ur	69.9	70.0	69.4	70.1	69.5
en+ur	ur	69.8	69.9	69.3	70.1	69.6

TABLE 3.5: Neuron fine-tuning results for Llama 3.1 on XNLI, for LAPE and Act Prob 90p. FTL is the fine-tuning language (English, the target, or both); EL is the evaluation language; task training data is always English. Each fine-tuned model is also evaluated under the test-time interventions (mean, $P90$, zero, $P10$). The best value in each row is in bold. No Int is the best column in nearly every row, and the FTL setting has little effect, so fine-tuning the identified neurons does not improve cross-lingual transfer.

The scores are close to those from fine-tuning the language-specific neurons in Table 3.5. For Llama 3.1, random-neuron fine-tuning gives 80.1 on Vietnamese, 74.8 on Hindi, and 69.8 on Urdu, which are within a fraction of a point of the corresponding language-specific runs. If the identified neurons carried information that ordinary neurons did not, fine-tuning them should beat fine-tuning random ones, and we do not see that gap. This is what we would expect if the selected neurons are not doing anything special for the task that other neurons cannot also do, and it fits the overlap and polysemanticity results discussed earlier. The control therefore supports the same conclusion as the rest of the section: the negative result is not because we picked the wrong neurons, but because acting on a small set of neurons, identified or random, does not change cross-lingual transfer.

Model	FTL	EL	No Int
Llama 3.1	rand	vi	80.1
	rand	hi	74.8
	rand	ur	69.8
Mistral Nemo	rand	vi	80.4
	rand	hi	75.6
	rand	ur	65.5

TABLE 3.6: Random-neuron control. Zero-shot (No Int) XNLI accuracy after fine-tuning ten randomly selected neurons per layer, instead of the language-specific neurons, for Llama 3.1 and Mistral Nemo. The scores are close to the language-specific fine-tuning results in Table 3.5, so the identified neurons are not doing something other neurons cannot.

3.8 Discussion

The results are consistent in their conclusion across tasks, models, and identification methods: interventions on language-specific neurons do not give a reliable improvement in cross-lingual task performance. This section discusses why.

3.8.1 Polysemantic Neurons and Entangled Function

The neuron analysis points to one explanation. The identified neurons are shared across languages (Section 3.6.2), and the random-neuron control (Section 3.7.4) gives almost the same scores as fine-tuning the identified neurons, so the selected neurons are not doing anything that other neurons cannot. This fits the idea that a single neuron takes part in more than one function, often called polysemanticity [Elhage et al., 2022]. If a neuron contributes to several computations at once, including ones the task depends on, then fixing its activation or restricting fine-tuning to it disturbs those other computations as well. The intervention is not surgical, even though the neuron was selected as language-specific.

3.8.2 Zero Is Not Deactivation

A common way to test a neuron’s role is to set its activation to zero and see what breaks. Our results show this is not a safe assumption here. Setting the activations to zero does not produce a large perplexity change (Section 3.6.4), and the Int 0 column does not collapse task performance the way removing a needed component would (Section 3.7.2); on XQuAD it is sometimes among the better intervention columns rather than the worst. For these activations, zero is just another value the neuron can take, not an off state. Conclusions that rely on zeroing a neuron to measure its importance should be treated with care.

3.8.3 Neurons Are Not Independent Across Languages

The overlap between the language sets (Section 3.6.2) and the off-diagonal perplexity changes (Section 3.6.4) both show that the neurons for one language are tied to those of others. This matters for the original goal. The appeal of language-specific neurons was that one could improve a target language by acting on its own neurons. If those neurons are shared with other languages, there is no clean handle to act on, and an edit aimed at one language reaches the others as well.

3.8.4 The Result Does Not Depend on How Neurons Are Selected

LAPE and Act Prob 90p score and select neurons differently. They also place the selected neurons in different parts of the model: as the layer-wise and overlap results show (Section 3.6.3, Section 3.6.2), LAPE concentrates neurons in the upper layers while Act Prob 90p places them at the bottom for Llama 3.1 and in the middle for Mistral Nemo, and the two methods select largely different sets. Despite this, they lead to the same downstream conclusion in every experiment. The fact that two methods which disagree on which neurons are language-specific, and where they sit, still produce the same outcome makes it unlikely that the negative result is an artifact of one selection rule or of the language-set normalization that LAPE depends on. It points instead to a property of how language is represented in these models, rather than of how we pick the neurons.

3.9 Limitations

This study has several limitations. It looks only at neurons in the MLP layers and does not consider the attention mechanism, which may also carry language-specific behaviour. The experiments use a small number of languages and two datasets, so the findings may not extend to all languages or tasks. The test-time interventions use statistics computed from Wikipedia text, which may not match how neurons behave on task inputs. The fine-tuning experiments are run on Llama 3.1, so the fine-tuning conclusions are not separately confirmed on Mistral Nemo. Finally, the study uses LoRA-based fine-tuning and does not explore other fine-tuning methods that might interact with the neurons differently. These points mark where the conclusion is and is not expected to hold.

Chapter 4

Current Research

Chapter 5

Group Robust Preference Optimization in the Multi-Label Setting

5.1 Introduction

5.1.1 Motivation

In multi-label classification a single input can carry several labels at once, and a model predicts each of them. The usual way to train such a model is to minimize one aggregate loss summed over all labels, which treats every label as equally important [Zangari et al., 2024; Schietgat et al., 2010]. The labels are rarely equal in practice. Some are rare, so their training signal is small next to the frequent ones [Charte et al., 2015b]. Some matter more than others, such as a rare disease against a common finding [Rajpurkar et al., 2017]. And some are simply harder to tell apart from a close alternative. With an aggregate loss, the easy and frequent labels take over training, and the labels that matter most are often the ones the model handles worst.

The standard fixes do not close this gap. Binary cross-entropy scores each label on its own, so the signal for a rare label is easily drowned out by the frequent ones [Ruby and Yendapalli, 2020; Charte et al., 2015b]. Focal Loss down-weights easy examples, but it still scores each label in isolation [Lin et al., 2018b]. None of these teaches the model to separate a true label from the specific incorrect label it is being confused with. What is missing is an objective that can express a preference between two labels, so that the model learns to score a true label above the competitor most likely to be mistaken for it. This is the gap the chapter addresses.

This chapter also fits the larger argument of the thesis. The earlier chapters showed that acting on a localized part of a model does not reliably change its behaviour, and that the training objective is the more effective place to work. Here we follow that idea in a different setting. Rather than analysing an objective, we design one: an objective that improves a chosen group of labels while protecting the rest.

5.1.2 Problem Statement

We study how to improve a model on a chosen subset of labels without degrading the others. Concretely, we split the label set into a privileged group, which we want to improve, and a non-privileged group, whose performance we want to keep at a baseline level, and we ask for a training objective that handles both at once. This breaks into three parts.

1. **How to improve the privileged labels.** The difficulty for these labels is usually fine-grained discrimination against a close, incorrect alternative, which a point-wise loss does not target. We need an objective that acts on pairs of labels and prefers the true one over its confusing competitor.
2. **How to protect the non-privileged labels.** Improving one group should not come at the cost of the rest. We need an objective for the non-privileged group that allows learning to focus elsewhere but steps in when a label starts to drop below a reliable baseline.
3. **How to balance the two.** The two objectives compete, and a fixed weighted sum does not adapt as training proceeds. We need a way to balance them that shifts focus to whichever group is currently doing worse.

5.1.3 Contributions

This chapter, based on our published work [Mondal et al., 2025a], reports the following.

- We propose FairPO, a framework for multi-label classification that partitions the labels into a privileged group and a non-privileged group and applies a different objective to each: a preference-based objective for the privileged group and a constrained objective for the non-privileged group.
- We adapt preference optimization, which was developed for aligning generative models, to a discriminative classification task. For a privileged label, FairPO finds the incorrect labels that are scored too close to it and trains the model to prefer the true label over them. We give three variants, based on DPO [Rafailov et al., 2023], CPO [Xu et al., 2024a], and SimPO [Meng et al., 2024b].
- We balance the two group objectives with a Group Robust Preference Optimization formulation [Ramesh et al., 2024], set up as a minimax problem over the two label groups, which is a new use of this method in this setting.

On MS-COCO and NUS-WIDE, FairPO improves accuracy on the rare privileged labels, by up to 3.44% Δ mAP over the BCE-SFT baseline with its CPO variant, while leaving the frequent labels almost unchanged, and it does better than the baselines we compare against.

5.2 The FairPO Framework

5.2.1 Problem Setup

We work in a standard multi-label classification setting with a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, where $\mathbf{x}_i$ is an input and $y_i \in \{0, 1\}^{|\mathcal{T}|}$ is its label vector over a set of $|\mathcal{T}|$ labels. We train a model with parameters $\mathbf{w}$ that gives a per-label score $m(\mathbf{x}_i; \mathbf{w}_t)$ for each label $t \in \mathcal{T}$. The framework also uses a fixed reference model with parameters $\hat{\mathbf{w}}_t$, which in our experiments is a model trained with binary cross-entropy.

The idea behind FairPO is to split the label set $\mathcal{T}$ into two disjoint groups:

- a *privileged* group $\mathcal{P} \subset \mathcal{T}$, the labels we want to improve, and
- a *non-privileged* group $\bar{\mathcal{P}} = \mathcal{T} \setminus \mathcal{P}$, the remaining labels, whose performance we want to keep at least at a baseline level.

How the labels are assigned to the two groups is left open and can be chosen for the problem, for example by label frequency for class imbalance, or by domain importance. In this chapter we use class imbalance, so the privileged group is the set of least frequent labels. The exact partition is given in Section 5.3.

Figure 5.1 gives an overview of the framework: each sampled label is routed to a privileged or non-privileged path with its own objective, and a group-robust step balances the two.

5.2.2 Objective for the Privileged Group

For the privileged group, reweighting a standard loss is not enough. The main difficulty for these labels is usually not basic classification but telling a label apart from a close, incorrect alternative. We therefore set up the objective for this group as a preference learning task.

Confusing counterparts. For an instance $\mathbf{x}_i$ and a privileged label $l \in \mathcal{P}$, we identify a set of confusing counterparts from the model’s current scores. There are two cases. When the true label is positive ($y_{il} = 1$), the confusing negatives are the incorrect labels that the model scores at least as high as the true label:

$$S_{il}^{\text{neg}} = \{k \in \mathcal{T} \mid y_{ik} = 0 \text{ and } m(\mathbf{x}_i; \mathbf{w}_k) \geq m(\mathbf{x}_i; \mathbf{w}_l)\}. \quad (5.1)$$

When the true label is negative ($y_{il} = 0$), the confusing positives are the correct labels that the model scores no higher than the true negative:

$$S_{il}^{\text{pos}} = \{k \in \mathcal{T} \mid y_{ik} = 1 \text{ and } m(\mathbf{x}_i; \mathbf{w}_k) \leq m(\mathbf{x}_i; \mathbf{w}_l)\}. \quad (5.2)$$

The full confusing set for (i, l) is $S_{il} = S_{il}^{\text{neg}} \cup S_{il}^{\text{pos}}$.

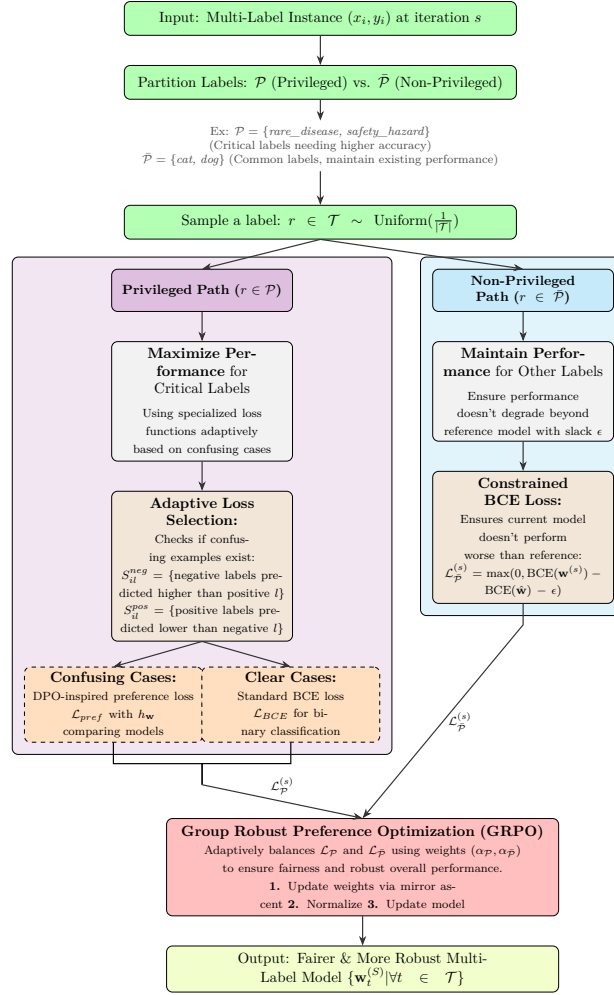


FIGURE 5.1: Overview of the FairPO framework. Labels are partitioned into a privileged group and a non-privileged group. A sampled label is routed to the privileged path, which uses a preference loss with a BCE fallback, or the non-privileged path, which uses a constrained BCE loss against a reference model. A Group Robust Preference Optimization step balances the two group losses.

Conditional objective with a fallback. The objective for the privileged group is adaptive. When S_{il} is non-empty, we apply a preference loss $\mathcal{L}_{\text{pref}}$ to the hard discriminative case. As the model improves, the confusing set for many instances becomes empty, and a preference loss alone would then give a sparse gradient and stall training. To keep a learning signal, we fall back to a standard binary cross-entropy loss when $S_{il} = \emptyset$:

$$\ell_{\text{BCE}}(\mathbf{x}_i, y_{il}; \mathbf{w}_l) = -[y_{il} \log m(\mathbf{x}_i; \mathbf{w}_l) + (1 - y_{il}) \log(1 - m(\mathbf{x}_i; \mathbf{w}_l))]. \quad (5.3)$$

Early in training the model makes many mistakes, the confusing sets are large, and the preference loss is used often. As the preference loss resolves these cases, the fallback takes over and fine-tunes the easier examples. The total privileged loss $\mathcal{L}_{\mathcal{P}}$ is the expectation over this conditional choice.

DPO variant. To write the preference losses in one form, let $m(\mathbf{x}_i; \mathbf{w}_p)$ be the score of the preferred label in a pair and $m(\mathbf{x}_i; \mathbf{w}_d)$ the score of the dispreferred label. The roles follow the ground truth: if $y_{il} = 1$ and $k \in S_{il}^{\text{neg}}$, then the true label is preferred ($p = l$) and the confusing negative is dispreferred ($d = k$); if $y_{il} = 0$ and $k \in S_{il}^{\text{pos}}$, then the true negative is preferred ($p = l$) and the confusing positive is dispreferred ($d = k$). Following DPO [Rafailov et al., 2023], the preference loss is taken relative to the reference model:

$$\mathcal{L}_{\text{pref}}^{\text{DPO}}(\mathbf{x}_i, p, d) = -\log \sigma \left(\beta \left(\log \frac{m(\mathbf{x}_i; \mathbf{w}_p)}{m(\mathbf{x}_i; \hat{\mathbf{w}}_p)} - \log \frac{m(\mathbf{x}_i; \mathbf{w}_d)}{m(\mathbf{x}_i; \hat{\mathbf{w}}_d)} \right) \right). \quad (5.4)$$

The full DPO-based privileged loss combines this with the fallback:

$$\mathcal{L}_{\mathcal{P}}^{\text{DPO}} = \mathbb{E}_{(\mathbf{x}_i, l) \text{ s.t. } l \in \mathcal{P}} \left[\mathbf{1}_{S_{il} \neq \emptyset} \cdot \mathcal{L}_{\text{pref}}^{\text{DPO}}(\mathbf{x}_i, p, d) + \mathbf{1}_{S_{il} = \emptyset} \cdot \ell_{\text{BCE}}(\mathbf{x}_i, y_{il}; \mathbf{w}_l) \right], \quad (5.5)$$

where for a sampled privileged label l with a non-empty confusing set, a counterpart k is sampled from S_{il} and the pair (p, d) is set from (l, k) and the ground truth.

SimPO variant. This variant adapts SimPO [Meng et al., 2024b] to give a reference-free preference loss with a target margin $\gamma > 0$, applied to the per-label scores:

$$\mathcal{L}_{\text{pref}}^{\text{SimPO}}(\mathbf{x}_i, p, d) = -\log \sigma \left(\beta \log \frac{m(\mathbf{x}_i; \mathbf{w}_p)}{m(\mathbf{x}_i; \mathbf{w}_d)} - \gamma \right). \quad (5.6)$$

The log-ratio term measures the relative preference, and the margin $-\gamma$ pushes the model to separate the two scores by a set amount rather than only rank them correctly. This term replaces the DPO term in the conditional objective of Equation 5.5.

CPO variant. The last variant adapts CPO [Xu et al., 2024a], which is reference-free and adds a BCE regularizer to the preference term. When a confusing set exists, the loss is

$$\mathcal{L}_{\text{pref}}^{\text{CPO}}(\mathbf{x}_i, p, d, l) = -\log \sigma \left(\beta \log \frac{m(\mathbf{x}_i; \mathbf{w}_p)}{m(\mathbf{x}_i; \mathbf{w}_d)} \right) + \lambda_{\text{CPO}} \cdot \ell_{\text{BCE}}(\mathbf{x}_i, y_{il}; \mathbf{w}_l). \quad (5.7)$$

The first term is a margin-free preference objective, and the second, weighted by λ_{CPO} , is the BCE loss for the privileged label l , which keeps the model grounded in absolute scores. As in the DPO case, this loss is used when a confusing set is found, and the BCE fallback of Equation 5.3 is used otherwise.

The reason for using a preference loss at all is that point-wise losses like BCE and Focal Loss score each label on its own. They answer whether a label’s score is right in isolation, which gives only a weak signal when a true label at 0.8 sits just above a confusing label at 0.7. The preference loss instead asks whether the true label is scored clearly above its specific competitor, and the log-ratio gives a direct gradient that pushes the two scores apart.

5.2.3 Objective for the Non-Privileged Group

For the non-privileged group the goal is not to push performance up but to keep it from dropping. We use a constrained objective against the reference model. For a label $j \in \bar{\mathcal{P}}$ we take the BCE loss of Equation 5.3, and we penalize the model only when its loss is worse than the reference model's loss by more than a slack margin $\epsilon \geq 0$, through a hinge:

$$\mathcal{L}_{\bar{\mathcal{P}}} = \mathbb{E}_{(\mathbf{x}_i, j) \text{ s.t. } j \in \bar{\mathcal{P}}} [\max(0, \ell_{\text{BCE}}(\mathbf{x}_i, y_{ij}; \mathbf{w}_j) - \ell_{\text{BCE}}(\mathbf{x}_i, y_{ij}; \hat{\mathbf{w}}_j) - \epsilon)]. \quad (5.8)$$

The gradient is zero as long as a non-privileged label is doing about as well as the reference, and a signal appears only when the label drops below that threshold. This keeps the non-privileged group from degrading while the model spends its capacity on the privileged group.

5.2.4 Group-Robust Balancing

We now have two objectives that pull against each other: the preference-based loss $\mathcal{L}_{\mathcal{P}}$ for the privileged group and the constrained loss $\mathcal{L}_{\bar{\mathcal{P}}}$ for the non-privileged group. A fixed weighted sum of the two does not handle this well, since the right balance changes during training. We instead treat the two label groups as the groups in a Group Robust Preference Optimization problem [Ramesh et al., 2024] and write the objective as a minimax game:

$$\min_{\{\mathbf{w}_t\}} \max_{\alpha \in \Delta} [\alpha_{\mathcal{P}} \mathcal{L}_{\mathcal{P}} + \alpha_{\bar{\mathcal{P}}} \mathcal{L}_{\bar{\mathcal{P}}}], \quad (5.9)$$

where $\alpha = (\alpha_{\mathcal{P}}, \alpha_{\bar{\mathcal{P}}})$ are adaptive weights on the probability simplex Δ . The inner maximization puts more weight on whichever group currently has the higher loss, so training focuses on the worse-off group, and the outer minimization updates the model under that weighting. This gives a way to manage the trade-off between the two groups rather than fixing it in advance.

5.2.5 Optimization

We solve Equation 5.9 with alternating mirror ascent on the weights α and mirror descent on the parameters $\mathbf{w}$, shown in Algorithm 1. One detail matters for stability. The two group losses can be on different scales, so using their raw values in the mirror ascent step makes the weight updates unstable. We instead update α from the change of each group's loss relative to its running average, which keeps the balancing step stable.

For the SimPO and CPO variants the structure is identical; only the preference term inside the privileged loss changes, to Equation 5.6 or Equation 5.7. The group-robust balancing and the non-privileged handling stay the same.

Algorithm 1 FairPO Training Overview (DPO-inspired)

-
- 1: **Initialize:** Model parameters $\{\mathbf{w}_t | t \in \mathcal{T}\}^{(0)}$ (e.g., from supervised fine-tuning), group weights $\alpha_{\mathcal{P}}^{(0)}, \alpha_{\bar{\mathcal{P}}}^{(0)}$
 - 2: **Input:** Dataset $\mathcal{D}$, reference parameters $\{\hat{\mathbf{w}}_t | t \in \mathcal{T}\}$, hyperparameters $\beta, \epsilon, \eta_{\mathbf{w}}, \eta_{\alpha}$.
 - 3: **For** each training iteration $s = 0, \dots, S - 1$:
 - 4: Sample instance $(x_i, y_i) \sim \mathcal{D}$ and a label $r \in \mathcal{T}$.
 - 5: **If** $r \in \mathcal{P}$:
 - 6: Compute privileged loss $\mathcal{L}_{\mathcal{P}}^{(s)}$ for (x_i, r) (Eq. 5.5, or BCE Eq. 5.3).
 - 7: **Else if** $r \in \bar{\mathcal{P}}$:
 - 8: Compute non-privileged loss $\mathcal{L}_{\bar{\mathcal{P}}}^{(s)}$ for (x_i, r) (Eq. 5.8).
 - 9: Update group weights $\alpha^{(s+1)}$ via mirror ascent using $\mathcal{L}_{\mathcal{P}}^{(s)}, \mathcal{L}_{\bar{\mathcal{P}}}^{(s)}$.
 - 10: Update model parameters $\{\mathbf{w}_t | t \in \mathcal{T}\}^{(s+1)}$ via mirror descent using the weighted gradients.
 - 11: **End For**
 - 12: **Return** Optimized parameters $\{\mathbf{w}_t | t \in \mathcal{T}\}^{(S)}$.
-

5.3 Experimental Setup

5.3.1 Datasets and Label Partition

We evaluate FairPO on two multi-label image benchmarks, MS-COCO 2014 [Lin et al., 2015b] and NUS-WIDE [Chua et al., 2009b]. MS-COCO has 80 object categories, with 82,783 training images and 40,504 validation images, which we use as the test set. NUS-WIDE has 81 concept labels, with 161,789 training images and 107,859 test images under the common split.

To set up the class-imbalance case, we partition the labels by frequency. On each dataset the privileged group $\mathcal{P}$ is the set of 16 least frequent labels, which is about 20% of the labels, and the non-privileged group $\bar{\mathcal{P}}$ is the rest, 64 labels on MS-COCO and 65 on NUS-WIDE. This makes the rare labels the ones we try to improve and the frequent labels the ones we try to maintain. The partition is one choice; as noted in Section 5.2.1, the framework allows other criteria, such as domain importance.

5.3.2 Model

The base model is a Vision Transformer, `vit-base-patch16-224` [Dosovitskiy et al., 2021b], pre-trained on ImageNet-21k and fine-tuned on ImageNet-1k. We keep the backbone frozen except for its final encoder layer, which we leave trainable so the higher-level features can adapt. On top of the backbone, each label has its own non-linear MLP head that takes the 768-dimensional feature from the ViT [CLS] token and outputs a single logit, passed through a sigmoid to give the per-label score $m(\mathbf{x}_i; \mathbf{w}_t)$. Each head has the layer sizes $768 \rightarrow 256 \rightarrow 64 \rightarrow 16 \rightarrow 4 \rightarrow 1$ with ReLU activations between layers, and the heads are independent across labels.

5.3.3 Baselines and Variants

We compare against three baselines. BCE-SFT trains the same model with binary cross-entropy; this is also the reference model $\hat{\mathbf{w}}$ used by the FairPO objectives. Group DRO + BCE [Sagawa et al., 2020b] applies distributionally robust optimization over label groups with a BCE loss. Focal Loss [Lin et al., 2018b] down-weights easy examples. We report the three FairPO variants from Section 5.2: FairPO-DPO, FairPO-SimPO, and FairPO-CPO.

5.3.4 Metrics and Implementation

We report mean average precision (mAP), Sample F1, and Accuracy, each split into the privileged group $\mathcal{P}$ and the non-privileged group $\bar{\mathcal{P}}$. The main quantity of interest is $\Delta\text{mAP}(\mathcal{P})$, the change in mAP on the privileged group relative to a baseline. All trainable parameters are updated with AdamW [Loshchilov and Hutter, 2019]. We train for at most 25 epochs with a batch size of 32, with early stopping on validation mAP using a patience of 5 epochs. Images are resized to 224×224 , normalized with ImageNet statistics, and augmented with random horizontal flips and random resized crops. Each result is the mean over three runs.

5.4 Results

Tables 5.1 and 5.2 report the results on MS-COCO and NUS-WIDE. Across both datasets, FairPO-CPO gives the best performance on the privileged group. On MS-COCO it improves $\Delta\text{mAP}(\mathcal{P})$ by +1.41 over the strongest baseline (Focal Loss) and by +3.44 over BCE-SFT, and the gain over the baselines is statistically significant. At the same time the drop on the non-privileged group is small and not statistically significant (-0.31 relative to BCE-SFT). The same pattern holds on NUS-WIDE, where FairPO-CPO gives the best privileged-group mAP with a non-privileged drop that is again small and not significant. This is the behaviour the framework is designed for: the model shifts capacity towards the rare labels without measurably hurting the frequent ones.

The three variants do not behave the same way. FairPO-DPO improves the privileged group but with a larger non-privileged drop than CPO. FairPO-SimPO is the weakest of the three, falling below the baselines on the privileged group on both datasets. FairPO-CPO is the most effective. The likely reason is its design: unlike DPO it is reference-free, and unlike SimPO it includes a BCE regularizer that gives the model a signal for absolute correctness alongside the relative preference. Learning both the relative ranking and the absolute score makes its training more stable and is what makes the CPO variant the most practical choice.

TABLE 5.1: Performance comparison on MS-COCO. $\mathcal{P}$ is the privileged label set (20% least frequent), and $\bar{\mathcal{P}}$ is the non-privileged set (remaining 80%). Results are mean $\pm$ std. dev. over 3 runs. The best result for each metric is in **bold**. Δ mAP is computed relative to the strongest baseline for each group (Focal Loss for $\mathcal{P}$, BCE-SFT for $\bar{\mathcal{P}}$). † marks a statistically significant improvement ($p < 0.05$); ‡ marks a difference that is not significant ($p > 0.1$).

Method	mAP		Sample F1		Accuracy		Δ mAP ($\mathcal{P}$) (vs. Focal)	Δ mAP ($\bar{\mathcal{P}}$) (vs. BCE)
	$\mathcal{P}$	$\bar{\mathcal{P}}$	$\mathcal{P}$	$\bar{\mathcal{P}}$	$\mathcal{P}$	$\bar{\mathcal{P}}$		
BCE-SFT [Ruby and Yendapalli, 2020]	86.32 $\pm$ 0.11	90.65 $\pm$ 0.08	61.43 $\pm$ 0.15	64.89 $\pm$ 0.12	94.89 $\pm$ 0.09	98.12 $\pm$ 0.05	-2.03	Ref
GDRO + BCE [Sagawa et al., 2020b]	87.92 $\pm$ 0.12	90.41 $\pm$ 0.09	62.31 $\pm$ 0.20	64.75 $\pm$ 0.13	95.72 $\pm$ 0.10	98.05 $\pm$ 0.05	-0.43	-0.24 ‡
Focal Loss [Lin et al., 2018b]	88.35 $\pm$ 0.14	89.81 $\pm$ 0.12	63.15 $\pm$ 0.16	64.18 $\pm$ 0.15	96.11 $\pm$ 0.12	97.90 $\pm$ 0.07	Ref	-0.84
FairPO-DPO	88.02 $\pm$ 0.15	89.97 $\pm$ 0.11	63.45 $\pm$ 0.17	63.65 $\pm$ 0.16	97.89 $\pm$ 0.13	98.04 $\pm$ 0.06	-0.33	-0.68
FairPO-SimPO	87.67 $\pm$ 0.18	88.76 $\pm$ 0.21	62.34 $\pm$ 0.22	63.12 $\pm$ 0.19	95.69 $\pm$ 0.15	97.78 $\pm$ 0.09	-0.68	-1.89
FairPO-CPO	89.76 $\pm$ 0.09	90.34 $\pm$ 0.07	64.01 $\pm$ 0.13	64.32 $\pm$ 0.10	98.03 $\pm$ 0.07	98.06 $\pm$ 0.05	+1.41 †	-0.31 ‡

TABLE 5.2: Performance comparison on NUS-WIDE. Notation is the same as Table 5.1.

Method	mAP		Sample F1		Accuracy		Δ mAP ($\mathcal{P}$) (vs. Focal)	Δ mAP ($\bar{\mathcal{P}}$) (vs. BCE)
	$\mathcal{P}$	$\bar{\mathcal{P}}$	$\mathcal{P}$	$\bar{\mathcal{P}}$	$\mathcal{P}$	$\bar{\mathcal{P}}$		
BCE-SFT [Ruby and Yendapalli, 2020]	63.53 $\pm$ 0.15	70.24 $\pm$ 0.11	48.12 $\pm$ 0.21	55.83 $\pm$ 0.14	91.51 $\pm$ 0.12	95.22 $\pm$ 0.08	-2.28	Ref
GDRO + BCE [Sagawa et al., 2020b]	64.84 $\pm$ 0.16	69.91 $\pm$ 0.12	49.13 $\pm$ 0.22	55.62 $\pm$ 0.15	92.11 $\pm$ 0.13	95.13 $\pm$ 0.09	-0.97	-0.33
Focal Loss [Lin et al., 2018b]	65.81 $\pm$ 0.17	68.95 $\pm$ 0.15	50.33 $\pm$ 0.20	54.31 $\pm$ 0.18	93.05 $\pm$ 0.11	94.75 $\pm$ 0.11	Ref	-1.29
FairPO-DPO	66.34 $\pm$ 0.20	69.05 $\pm$ 0.14	51.71 $\pm$ 0.25	54.52 $\pm$ 0.19	93.92 $\pm$ 0.16	95.04 $\pm$ 0.09	+0.53	-1.19
FairPO-SimPO	64.11 $\pm$ 0.22	68.03 $\pm$ 0.24	48.82 $\pm$ 0.28	53.81 $\pm$ 0.22	91.94 $\pm$ 0.18	94.52 $\pm$ 0.13	-1.70	-2.21
FairPO-CPO	67.12 $\pm$ 0.14	69.83 $\pm$ 0.10	52.21 $\pm$ 0.19	55.24 $\pm$ 0.13	94.31 $\pm$ 0.10	95.12 $\pm$ 0.08	+1.31 †	-0.41 ‡

5.5 Ablation Studies

We run ablations on MS-COCO with FairPO-CPO to check that each part of the framework matters. Tables 5.3 and 5.4 report the results, with Δ mAP($\mathcal{P}$) measured against BCE-SFT and the parenthesized values showing the change relative to the full FairPO-CPO model.

Table 5.3 removes one component at a time. Replacing the preference loss with BCE drops the privileged gain from +3.44 to +1.80, so the preference signal accounts for much of the improvement. Removing the non-privileged constraint keeps the privileged gain close (+3.23) but lets the non-privileged group degrade, which defeats the purpose of the framework. Fixing the group weights at 0.5/0.5 instead of using the adaptive balancing lowers the privileged gain to +2.16, showing that the balancing step is doing real work. A Global CPO variant that applies the preference loss to all labels with no privileged/non-privileged split reaches only +2.23, which is better than BCE but well short of the full model, so the targeted partition matters.

Table 5.4 looks at the preference objective itself. Restricting the preference signal to only confusing negatives, and dropping the confusing positives, is the most damaging change: the privileged gain falls to -13.17, a large drop, because the model loses the signal that handles true-negative labels with confusing positives. Removing the BCE fallback for the non-confusing case lowers the gain from +3.44 to +2.73, which confirms that the fallback is needed to keep a stable signal once the confusing sets empty out. Taken together, the ablations show that the gain comes from the complete framework, not from any single piece.

TABLE 5.3: Ablation on the core components of FairPO-CPO (MS-COCO). $\Delta\text{mAP}(\mathcal{P})$ is measured against BCE-SFT. Parentheses show the change relative to the full FairPO-CPO model.

Method Variant	mAP		Sample F1		Accuracy		$\Delta\text{mAP}(\mathcal{P})$
	$\mathcal{P}$	$\bar{\mathcal{P}}$	$\mathcal{P}$	$\bar{\mathcal{P}}$	$\mathcal{P}$	$\bar{\mathcal{P}}$	
FairPO-CPO (Full)	89.76	90.34	64.01	64.32	98.03	98.06	+3.44
<i>w/o Preference Loss</i> ($\mathcal{L}_{\mathcal{P}}$ is BCE)	88.12 (-1.64)	90.45 (+0.11)	62.45 (-1.56)	64.80 (+0.48)	95.80 (-2.23)	98.09 (+0.03)	+1.80
<i>w/o $\bar{\mathcal{P}}$ Constraint</i> ($\mathcal{L}_{\bar{\mathcal{P}}}$ is BCE)	89.55 (-0.21)	88.98 (-1.36)	63.70 (-0.31)	62.95 (-1.37)	97.90 (-0.13)	97.55 (-0.51)	+3.23
<i>w/o GRPO</i> (Fixed 0.5/0.5 weights)	88.48 (-1.28)	89.75 (-0.59)	62.88 (-1.13)	63.50 (-0.82)	96.53 (-1.50)	97.88 (-0.18)	+2.16
<i>Global CPO (all labels)</i> (No $\mathcal{P}/\bar{\mathcal{P}}$ split)	88.55 (-1.21)	90.68 (+0.34)	62.75 (-1.26)	64.85 (+0.47)	96.95 (-1.08)	98.11 (+0.05)	+2.23

TABLE 5.4: Ablation on the preference formulation (FairPO-CPO, MS-COCO). $\Delta\text{mAP}(\mathcal{P})$ is measured against BCE-SFT. Parentheses show the change relative to the full FairPO-CPO model.

Preference Detail Variant	mAP		Sample F1		Accuracy		$\Delta\text{mAP}(\mathcal{P})$
	$\mathcal{P}$	$\bar{\mathcal{P}}$	$\mathcal{P}$	$\bar{\mathcal{P}}$	$\mathcal{P}$	$\bar{\mathcal{P}}$	
FairPO-CPO (Full) (Conf. Neg & Pos, BCE Fallback)	89.76	90.34	64.01	64.32	98.03	98.06	+3.44
<i>Only Confusing Negatives</i>	73.15 (-16.61)	90.25 (-0.09)	47.88 (-16.13)	64.20 (-0.12)	94.67 (-3.36)	98.01 (-0.05)	-13.17
<i>w/o BCE Fallback</i> (No loss if $S_{il} = \emptyset$)	89.05 (-0.71)	90.21 (-0.13)	63.20 (-0.81)	64.10 (-0.22)	97.55 (-0.48)	97.99 (-0.07)	+2.73

5.6 Discussion and Limitations

FairPO shows that an objective can be designed to improve a chosen group of labels while protecting the rest. The preference loss gives the privileged group a signal that point-wise losses do not, the constrained objective keeps the non-privileged group from degrading, and the group-robust balancing manages the trade-off between them. The results bear out the larger point of the thesis: the useful place to act is the training objective. Where Chapter 3 found that editing a small set of internal components did not reallocate a model’s behaviour, here a change at the level of the objective does, improving the rare labels without measurable harm to the frequent ones.

The framework has limitations. The confusing set is computed from the model’s current scores and changes during training, which can make it unstable. The DPO variant needs a reference model, which the CPO and SimPO variants avoid. The group-robust balancing has hyperparameters that need care, since the two losses are on different scales and the balancing step has to be stabilized through loss scaling. The experiments here use a frequency-based label partition on two image benchmarks; other partition criteria and other domains are left for future work, along with extending the approach beyond classification.

Chapter 6

Conclusions

6.1 Summary of Research Contributions and Progress

This report has documented a comprehensive and systematic research program aimed at addressing the critical challenges of enhancing and controlling Large Language Models for low-resource languages. The work has progressed methodically from foundational interpretability and advanced theoretical explorations to direct, practical application, yielding significant insights that have actively shaped the research trajectory.

The research journey commenced with a foundational investigation into the internal mechanisms of multilingual models. This work, presented at the NAACL 2024 Workshop [Mondal et al., 2025b], rigorously tested the hypothesis that language-specific neurons could be leveraged for cross-lingual transfer. The key finding that direct interventions on these neurons are insufficient for improving downstream task performance due to their polysemantic nature provided a critical, empirical justification for pivoting away from surgical internal edits and towards more holistic, output-level control mechanisms.

To build the necessary expertise for such control, a second foundational project was undertaken, culminating in a full paper submission to NeurIPS 2025 [Mondal et al., 2025a]. This research, *FairPO*, involved designing a novel, sophisticated framework for fair multi-label classification. It required moving beyond the standard application of preference optimization to adapt its core principles to a new domain, integrate them with group-robust optimization (GRPO), and handle dynamically generated preference sets. This project provided an indispensable, deep, and practical mastery of advanced model alignment frameworks.

The current research phase applies this expertise to the task of low-resource Neural Machine Translation. Preliminary experiments were conducted to fine-tune Llama 3.1 for English-to-Bengali translation using Direct Preference Optimization (DPO). The initial results were unexpected, showing a significant degradation in performance compared to both a one-shot and a standard fine-tuning baseline. This negative result is, in itself, a crucial scientific finding, highlighting the non-trivial complexities and potential instabilities of applying preference optimization

in this specific context. It has since prompted a necessary and rigorous debugging phase, which is currently underway.

6.2 Overall Thesis Contribution and Future Outlook

The primary contribution of the work to date is a multi-faceted investigation into model control for low-resource NLP. This includes a peer-reviewed contribution that challenges a prevailing hypothesis in interpretability, a conference-submitted paper that showcases mastery of advanced optimization, and a critical empirical finding that informs the practical application of DPO.

The future work is now well-defined and strategically planned. The immediate priority, is to systematically diagnose the issues encountered in the NMT experiments to produce a conclusive result. Following this consolidation phase, the research will advance to a novel and ambitious objective: the development of a *unified, agentic framework for low-resource QA and translation*.

This proposed framework represents a significant conceptual step forward, aiming to synthesize the strengths of several state-of-the-art paradigms. It will use the agentic, multi-step reasoning pipeline of *KGAREVION* [?] as its blueprint, adapt it for unstructured text retrieval, and instill robustness against noisy context by incorporating the training principles of *RAFT* [?]. Critically, the final generation module of this agent will be enhanced using the preference optimization techniques from Phase 1 of this research, creating a powerful synergy between factually grounded reasoning and high-quality linguistic generation.

References

- AI, M., NVIDIA, 2024. Mistral nemo. URL: <https://mistral.ai/news/mistral-nemo>.
- Artetxe, M., Ruder, S., Yogatama, D., 2020. On the cross-lingual transferability of monolingual representations, in: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, Association for Computational Linguistics. pp. 4623–4637.
- Bhattacharya, S., Bojar, O., 2023. Unveiling multilinguality in transformer models: Exploring language specificity in feed-forward networks. arXiv preprint arXiv:2310.15552 .
- Charte, F., Rivera, A.J., del Jesus, M.J., Herrera, F., 2015a. Addressing imbalance in multilabel classification: Measures and random resampling algorithms. *Neurocomputing* 163, 3–16.
- Charte, F., Rivera, A.J., del Jesús, M.J., Herrera, F., 2015b. Addressing imbalance in multilabel classification: Measures and random resampling algorithms. *Neurocomputing* 163, 3–16. URL: <https://api.semanticscholar.org/CorpusID:207107609>.
- Chua, T.S., Tang, J., Hong, R., Li, H., Luo, Z., Zheng, Y., 2009a. NUS-WIDE: A real-world web image database from national university of singapore, in: Proceedings of the ACM International Conference on Image and Video Retrieval (CIVR).
- Chua, T.S., Tang, J., Hong, R., Li, H., Luo, Z., Zheng, Y., 2009b. Nus-wide: a real-world web image database from national university of singapore. URL: <https://api.semanticscholar.org/CorpusID:6483070>.
- Conneau, A., Rinott, R., Lample, G., Williams, A., Bowman, S., Schwenk, H., Stoyanov, V., 2018. XNLI: Evaluating cross-lingual sentence representations, in: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Brussels, Belgium. pp. 2475–2485.
- Conneau, A., Wu, S., Li, H., Zettlemoyer, L., Stoyanov, V., 2020. Emerging cross-lingual structure in pretrained language models. arXiv preprint arXiv:1911.01464 .
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., Houlsby, N., 2021a. An image is worth 16x16 words: Transformers for image recognition at scale, in: International Conference on Learning Representations (ICLR).
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., Houlsby, N., 2021b.

- An image is worth 16x16 words: Transformers for image recognition at scale. URL: <https://arxiv.org/abs/2010.11929>, [arXiv:2010.11929](#).
- Durrani, N., Sajjad, H., Dalvi, F., Belinkov, Y., 2020. Analyzing individual neurons in pre-trained language models, in: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), Association for Computational Linguistics. pp. 4865–4880.
- Elhage, N., Hume, T., Olsson, C., Schiefer, N., Henighan, T., Kravec, S., Hatfield-Dodds, Z., Lasenby, R., Drain, D., Chen, C., Grosse, R., McCandlish, S., Kaplan, J., Amodei, D., Wattenberg, M., Olah, C., 2022. Toy models of superposition. Transformer Circuits Thread [ArXiv:2209.10652](#).
- Grattafiori, A., 2024. The llama 3 herd of models. [arXiv:2407.21783](#).
- Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., et al., 2025. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. [arXiv preprint arXiv:2501.12948](#).
- Guo, H., Wang, S., 2021. Long-tailed multi-label visual recognition by collaborative training on uniform and re-balanced samplings, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).
- Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., 2022. LoRA: Low-rank adaptation of large language models, in: International Conference on Learning Representations (ICLR). [ArXiv:2106.09685](#).
- Huo, J., Yan, Y., Hu, B., Yue, Y., Hu, X., 2024. MMNeuron: Discovering neuron-level domain-specific interpretation in multimodal large language model, in: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Miami, Florida, USA. pp. 6801–6816.
- Kojima, T., Okimura, I., Iwasawa, Y., Yanaka, H., Matsuo, Y., 2024. On the multilingual ability of decoder-based pre-trained language models: Finding and controlling language-specific neurons, in: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), Association for Computational Linguistics, Mexico City, Mexico. pp. 6919–6971.
- Kornblith, S., Norouzi, M., Lee, H., Hinton, G., 2019. Similarity of neural network representations revisited, in: Proceedings of the 36th International Conference on Machine Learning (ICML), PMLR. pp. 3519–3529.
- Lai, W., Hangya, V., Fraser, A., 2024. Style-specific neurons for steering LLMs in text style transfer, in: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Miami, Florida, USA. pp. 13427–13443.
- Lin, T.Y., Goyal, P., Girshick, R., He, K., Dollár, P., 2018a. Focal loss for dense object detection. [arXiv preprint arXiv:1708.02002](#).

- Lin, T.Y., Goyal, P., Girshick, R., He, K., Dollár, P., 2018b. Focal loss for dense object detection. URL: <https://arxiv.org/abs/1708.02002>, [arXiv:1708.02002](#).
- Lin, T.Y., Maire, M., Belongie, S., Bourdev, L., Girshick, R., Hays, J., Perona, P., Ramanan, D., Zitnick, C.L., Dollár, P., 2015a. Microsoft COCO: Common objects in context. arXiv preprint [arXiv:1405.0312](#) .
- Lin, T.Y., Maire, M., Belongie, S., Bourdev, L., Girshick, R., Hays, J., Perona, P., Ramanan, D., Zitnick, C.L., Dollár, P., 2015b. Microsoft coco: Common objects in context. URL: <https://arxiv.org/abs/1405.0312>, [arXiv:1405.0312](#).
- Liu, T., Wang, H., Wang, Y., Wang, X., Su, L., Gao, J., 2023. SimFair: A unified framework for fairness-aware multi-label classification. arXiv preprint [arXiv:2302.09683](#) .
- Loshchilov, I., Hutter, F., 2019. Decoupled weight decay regularization. URL: <https://arxiv.org/abs/1711.05101>, [arXiv:1711.05101](#).
- Lu, M., Zhang, R., Eickhoff, C., Pavlick, E., 2025. Paths not taken: Understanding and mending the multilingual factual recall pipeline. arXiv preprint [arXiv:2505.20546](#) .
- Meng, Y., Xia, M., Chen, D., 2024a. SimPO: Simple preference optimization with a reference-free reward, in: Advances in Neural Information Processing Systems (NeurIPS). [ArXiv:2405.14734](#).
- Meng, Y., Xia, M., Chen, D., 2024b. Simpo: Simple preference optimization with a reference-free reward. URL: <https://arxiv.org/abs/2405.14734>, [arXiv:2405.14734](#).
- Mondal, S.K., Chanda, P., Varmora, A., Ramakrishnan, G., 2025a. Fairpo: Robust preference optimization for fair multi-label learning. URL: <https://arxiv.org/abs/2505.02433>, [arXiv:2505.02433](#).
- Mondal, S.K., Sen, S., Singhanian, A., Jyothi, P., 2025b. Language-specific neurons do not facilitate cross-lingual transfer. URL: <https://aclanthology.org/2025.insights-1.6/>.
- nostalgebraist, 2020. Interpreting GPT: The logit lens. LessWrong. <https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens>.
- Petrov, A., La Malfa, E., Torr, P.H.S., Bibi, A., 2023. Language model tokenizers introduce unfairness between languages, in: Advances in Neural Information Processing Systems (NeurIPS).
- Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C.D., Finn, C., 2023. Direct preference optimization: Your language model is secretly a reward model, in: Advances in Neural Information Processing Systems (NeurIPS). [ArXiv:2305.18290](#).
- Rajpurkar, P., Irvin, J., Zhu, K., Yang, B., Mehta, H., Duan, T., Ding, D., Bagul, A., Langlotz, C., Shpanskaya, K., Lungren, M.P., Ng, A.Y., 2017. Chexnet: Radiologist-level pneumonia detection on chest x-rays with deep learning. URL: <https://arxiv.org/abs/1711.05225>, [arXiv:1711.05225](#).

- Ramesh, S.S., Hu, Y., Chaimalas, I., Mehta, V., Sessa, P.G., Bou Ammar, H., Bogunovic, I., 2024. Group robust preference optimization in reward-free RLHF, in: *Advances in Neural Information Processing Systems (NeurIPS)*. ArXiv:2405.20304.
- Ruby, U., Yendapalli, V., 2020. Binary cross entropy with deep learning technique for image classification. doi:[10.30534/ijatcse/2020/175942020](https://doi.org/10.30534/ijatcse/2020/175942020).
- Sagawa, S., Koh, P.W., Hashimoto, T.B., Liang, P., 2020a. Distributionally robust neural networks for group shifts: On the importance of regularization for worst-case generalization, in: *International Conference on Learning Representations (ICLR)*.
- Sagawa, S., Koh, P.W., Hashimoto, T.B., Liang, P., 2020b. Distributionally robust neural networks for group shifts: On the importance of regularization for worst-case generalization. URL: <https://arxiv.org/abs/1911.08731>, arXiv:1911.08731.
- Schietgat, L., Vens, C., Struyf, J., Blockeel, H., Kocev, D., Džeroski, S., 2010. Predicting gene function using hierarchical multi-label decision tree ensembles. *BMC Bioinformatics* 11, 2. URL: <https://doi.org/10.1186/1471-2105-11-2>, doi:10.1186/1471-2105-11-2.
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y.K., Wu, Y., Guo, D., 2024. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300 .
- Shazeer, N., 2020. GLU variants improve transformer. arXiv preprint arXiv:2002.05202 .
- Shi, F., Suzgun, M., Freitag, M., Wang, X., Srivats, S., Vosoughi, S., Chung, H.W., Tay, Y., Ruder, S., Zhou, D., Das, D., Wei, J., 2023. Language models are multilingual chain-of-thought reasoners, in: *The Eleventh International Conference on Learning Representations (ICLR)*. ArXiv:2210.03057.
- Tang, T., Luo, W., Huang, H., Zhang, D., Wang, X., Zhao, X., Wei, F., Wen, J.R., 2024. Language-specific neurons: The key to multilingual capabilities in large language models, in: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, Bangkok, Thailand. pp. 5701–5715.
- Wendler, C., Veselovsky, V., Monea, G., West, R., 2024. Do llamas work in English? on the latent language of multilingual transformers, in: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, Bangkok, Thailand. pp. 15366–15394.
- Wikimedia, . Wikimedia downloads. URL: <https://dumps.wikimedia.org>.
- Xie, W., Feng, Y., Gu, S., Yu, D., 2021. Importance-based neuron allocation for multilingual neural machine translation, in: *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, Association for Computational Linguistics. pp. 5725–5737.

- Xu, H., Sharaf, A., Chen, Y., Tan, W., Shen, L., Durme, B.V., Murray, K., Kim, Y.J., 2024a. Contrastive preference optimization: Pushing the boundaries of llm performance in machine translation. URL: <https://arxiv.org/abs/2401.08417>, [arXiv:2401.08417](#).
- Xu, H., Sharaf, A., Chen, Y., Tan, W., Shen, L., Van Durme, B., Murray, K., Kim, Y.J., 2024b. Contrastive preference optimization: Pushing the boundaries of LLM performance in machine translation. arXiv preprint arXiv:2401.08417 .
- Zangari, A., Marcuzzo, M., Rizzo, M., Giudice, L., Albarelli, A., Gasparetto, A., 2024. Hierarchical text classification and its foundations: A review of current research. URL: <https://www.mdpi.com/2079-9292/13/7/1199>, doi:[10.3390/electronics13071199](#).
- Zhang, X., Liang, Y., Meng, F., Zhang, S., Chen, Y., Xu, J., Zhou, J., 2024. Multilingual knowledge editing with language-agnostic factual neurons. arXiv preprint arXiv:2406.16416 .
- Zhao, X., Yoshinaga, N., Oba, D., 2024a. Tracing the roots of facts in multilingual language models: Independent, shared, and transferred knowledge. arXiv preprint arXiv:2403.05189 .
- Zhao, Y., Zhang, W., Chen, G., Kawaguchi, K., Bing, L., 2024b. How do large language models handle multilingualism? arXiv preprint arXiv:2402.18815 .
- Zhu, S., Pan, L., Li, B., Xiong, D., 2024. LANDeRMT: Detecting and routing language-aware neurons for selectively finetuning LLMs to machine translation, in: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Bangkok, Thailand. pp. 12135–12148.